# Group Project Milestone 3:

# Integration and Communication — Deployment, Final Report and Presentation

Predicting Delivery Performance and Satisfaction Risk
on the Olist Brazilian E-Commerce Marketplace

Two prediction moments, one decision chain, and a deployed application


**Group 10**

| | |
|---|---|
| Pan Pinyou | A0329105L |
| Li Yudan | A0330335R |
| Zhu Gengyu | A0333773Y |

National University of Singapore


IT5006 Fundamentals of Data Analytics

AY2026/2027 Semester 1 · Final report, November 2026

**Application:** `https://team10-it5006-olist.streamlit.app`

**Repository:** `https://github.com/gengyuzccc-cell/Team10_IT5006_Ecommerce_Analytics_AY2627Sem1`

# Contents

**Executive Summary** **3**

**1 Problem Definition and Stakeholder Context** **4**

**2 Literature Review** **5**

**3 Dataset and Preprocessing** **8**

**4 Exploratory Analysis Highlights** **9**

**5 Feature Engineering** **11**

**6 Modelling Methodology** **12**

**7 Results and Evaluation** **13**

**8 Model Deployment and Application** **17**

**9 Business Recommendations and Impact** **19**

**10 Limitations and Conclusion** **21**

**A Reproducing this report** **28**

**B Data model, cleaning decisions and dictionary** **28**

**C Success criteria registered in advance** **33**

**D Hyperparameter search spaces and selected values** **33**

**E Re-checking a Phase 1 claim about least squares** **35**

**F Full result tables** **36**

**G The cross-validation protocol, and every candidate on every window** **36**

**H Lateness anatomy: where the late deliveries came from** **37**

**I Responses to the Phase 1 and Phase 2 feedback** **39**

**J Training, validation and test side by side** **40**

**K Feature catalogue** **41**

**L The outreach pilot** **42**

**M When is the review written?** **43**

**N Contributions** **44**

**O Figures** **45**

**AI Tool Declaration** **57**

# Executive Summary

Olist is a Brazilian marketplace that places small sellers on large retail platforms and manages fulfilment for them; its public dataset — 99,441 orders in nine relational tables, 2016–2018 — records each order from checkout to delivery to review. This project asks two questions of it, at two moments in an order's life, and turns the answers into decisions. **Problem 1, at checkout:** how long will this order take, and will it miss the date the customer was promised? **Problem 2, at delivery:** will this parcel draw a one- or two-star review? The first serves the operations team, which can re-quote a promise, warn a buyer or expedite a parcel; the second serves customer experience, which can reach a customer before the review is written. The two are halves of one chain: a late delivery raises the probability of a low review from 9.2% to 54.0%, the strongest relationship in the data.

Three disciplines run through the work. *Leakage is defined per prediction moment*, and a guard enforces the right feature set before any model is fitted — one forbidden column lifts the late flag's ROC-AUC from 0.589 to 1.000. *Time is respected*: the split is chronological and customer-disjoint, cross-validation uses expanding windows, and the test window (July–August 2018) was scored once, for models already chosen. *Complexity has to earn its place*: three model families — linear, tree-based, ensemble — are run as ladders from their simplest variant up, and every rung is compared with the one below it.

The lead-time regression reaches a mean absolute error of **3.90 days**, 41% below a constant fitted on the past and 60% below the platform's own promised date. Thresholding that forecast against the promise gives a late-order flag at 2.9 times the no-skill floor, where the best classifier trained on the binary label reaches 1.2 times. The low-review model reaches ROC-AUC 0.749 and PR-AUC 0.414, **3.75 times its floor**, on a target where always predicting "satisfied" scores 89% accuracy and catches nothing. Read as a ranked list, which is how a team with limited capacity uses it, contacting the riskiest 5% of deliveries reaches a would-be low reviewer two times in three.

Three recommendations follow, each with a stated cost. Operations should re-quote every promise with the model's estimate and pre-warn buyers whose late risk exceeds 11%, both nearly free; it should *not* expedite on the flag, whose precision never clears the 0.30 a R$12 intervention would need. Customer experience should work each day's deliveries from the top of the review-risk list to its capacity: at illustrative costs of R$5 per contact and R$40 per low review the policy is worth up to R$1,466 per thousand delivered orders, or R$775 counting only customers who have not yet written — for a late parcel 87% of low reviews precede delivery, so its buyer is reached on the promised date instead. Because that figure assumes a contact prevents the review it reaches — something no historical data can show — the report sizes the pilot that would test it: a randomised hold-out inside the daily list, about six weeks at ten contacts a day, with break-even at roughly one averted review in five.

Everything is deployed and reproducible. A Streamlit application at `https://team10-it5006-olist.streamlit.app` serves the three models on single orders and batch files, returns the ranked triage list, replays any held-out test day with its outcomes revealable and states in a model card which estimators are deployed. The repository holds the source tables, ten executed notebooks, the fitted artifacts and the scripts that regenerate every number in this document; a suite of 60 validation checks ties the metadata, the pickled models, the result tables and this text to one another. Both rounds of evaluator feedback are incorporated throughout and answered point by point in Appendix I.

# 1 Problem Definition and Stakeholder Context

## 1.1 Two problems, one decision chain

The exploratory analysis of Section 4 left one finding standing above the rest: a late delivery raises the probability of a 1–2 star review from 9.2% to 54.0%, a 5.9-fold increase monotone in how late the parcel is. The project turns that sentence into a pair of predictions placed at two points in an order's life (Table 1).

**Table 1:** The two problems. They share one feature pipeline, one model-family budget and one dataset; what separates them is *when* the model runs.

| | **Problem 1 — delivery performance** | **Problem 2 — satisfaction risk** |
|---|---|---|
| Question | How long will this order take, and will it miss its promise? | Will this delivered order draw a 1–2 star review? |
| Moment | `at_checkout` — the order has just been placed | `at_delivery` — the parcel has arrived, the review is not yet written |
| Stakeholder | Operations and logistics | Customer experience |
| Task | **1A regression**: lead time in days, on MAE. **1B classification**: delivered after the promised date, on PR-AUC against its no-skill floor | **Classification**: `review_score` $\leq 2$, on PR-AUC against its floor and balanced accuracy |
| Action | Re-quote the promise, warn the buyer, expedite freight | Reach out before the review lands |

The brief permits two problems or one framed twice; we take two, because they are halves of one decision chain: operations acts before the parcel ships, customer experience after it arrives and before the customer writes anything, and Problem 1's output is, in substance, Problem 2's most important input. Problem 1 is the brief's *dual framing*: one underlying quantity, two outputs, so the late-order flag *is* the regression's answer compared against the promised date; five classifiers trained directly on the binary label run alongside as a control arm (Section 7.2).

## 1.2 How the stakeholders act on the output

A prediction is only worth producing if someone changes a decision because of it — the lesson of the prescriptive delivery framework reviewed in Section 2, where the savings came from routing machine-learned delivery outcomes into a scheduling decision [23]. Table 2 states what each team does with each output and what decides whether the action pays, and it shapes the evaluation: the cheap operations levers apply to every order and are judged on the regression's accuracy; the expensive one, and all of customer experience's work, are rationed by capacity and are judged on what the top-$K$ of a ranked list delivers (Section 7.4).

## 1.3 Targets, and what the review score measures

All three targets come from the provided tables alone. *Lead time* is the time from purchase (`order_purchase_timestamp`) to arrival (`order_delivered_customer_date`), in days, defined for the 97.0% of orders that reached the customer. *Late* is delivery strictly after `order_estimated_delivery_date`, which carries no time component, so lateness is accurate to about a day. *Low review* is `review_score` $\leq 2$, defined for the 99.2% carrying a review. Undefined rows are `NaN` and excluded, never imputed to a negative label: the

**Table 2:** What each team does with the predictions, and what decides whether the action pays. Costs are the stated assumptions of Section 9.2.

| Who, when | Lever | Cost | Trigger, and the evidence behind it |
|---|---|---|---|
| Operations, at checkout | Re-quote the promised date with the model's estimate | none | every order; MAE 3.90 days against the promise's 9.79 (Section 7.1) |
| | Pre-warn the buyer | near zero | $p_{\text{late}} \geq 0.11$ catches 82.6% of the orders that will miss |
| | Expedite freight | R$12 | only where precision clears 0.30, which it never does (Section 7.4); manual review of the top of the list |
| | Monitor sellers, routes, carrier | analyst time | track-record features and the weekly late series (Figure 23) as early warning |
| Customer experience, at delivery | Contact the customer before the review is written | R$5 | the day's deliveries ranked by $p_{\text{low}}$, worked top-down to capacity $K$; at 5% of deliveries, two contacts in three reach a would-be 1–2 star reviewer |

event has not happened rather than gone missing, and a silent customer is not a satisfied one.

The target of Problem 2 is a platform rating, not the *perceived quality* of the survey literature, and the distinction — which the Phase 1 evaluation asked for — matters for what the model claims. Perceived service quality is measured by instruments such as E-S-QUAL [35] and e-LSQ [39]: self-reported and retrospective. A star rating is a behavioural trace left by the customers who choose to write one, and its J-shaped distribution — 57.8% five stars here, with a second mode at one star — is the signature of acquisition and under-reporting biases [20, 21]; a rating also moves with the product, its price and the seller's communication [10]. Problem 2 therefore predicts the *rating*, the observable the customer-experience team is measured on, and claims nothing about latent satisfaction; the consequence, that part of the residual is unobserved product and seller effects, is carried into Section 10.

### 1.4 Success criteria, registered in advance

Expected ranges for all six headline metrics were written into `config.EXPECTED_BANDS` *before* any model was tuned, from the Phase 1 results and untuned scoping probes (Appendix C). They are the targets the work is judged against and they are a leak alarm: here the signature of leakage is not a subtle improvement but an implausible one (Section 3.3), so a result landing far *above* its band is investigated before it is reported. All six landed inside; none above. The ceiling was stated as plainly as the target: the largest swings in lateness are shocks to carrier capacity and to the platform's own promising policy (Section 4.2), none of which is an attribute of an order, so a late-delivery classifier reporting 95% accuracy here would be the majority-class rule or a leak.

## 2 Literature Review

This review was revised after the Phase 1 evaluation, which asked for four things the first version lacked: a distinction between survey-based perceived quality and platform ratings, applied rather than foundational citations for the model choices with a justification within the tree family, literature support for the features, and an evaluation aligned with a team's

limited capacity. Each is addressed in the subsection it belongs to.

## 2.1 Fulfilment, delivery promises and satisfaction

The e-commerce service-quality literature is direct about which lever matters. Parasuraman et al. [35] decompose electronic service quality into efficiency, *fulfilment*, system availability and privacy, and find fulfilment — the accuracy of delivery promises and the condition in which goods arrive — among the strongest contributors to perceived quality; Rao et al. [39] sharpen this into logistics terms across 260 online retailers. Both measure a survey construct. What a marketplace observes is a rating, and ratings are not surveys: they are written by a self-selected minority, J-shaped by acquisition and under-reporting biases [20, 21], and moved by product, price and seller behaviour as much as by service [10]. Transactional evidence bridges the two: across more than 15 million marketplace orders, Deshpande and Pendem [11] trace a chain — logistics performance drives the rating a customer leaves, and that rating moves seller sales. That chain makes a low-review model a commercial instrument rather than a diagnostic, and the rating is the construct this project predicts.

The promise itself is a second decision variable. Cui et al. [9] show that committing to a faster delivery date raises sales but increases returns and depresses retention, so an estimated delivery date is a business decision, not a forecast, and should not be expected to behave like one. Prediction pays only when it feeds a decision: Kandula et al. [23] route machine-learned delivery outcomes into a scheduling optimiser and report cost savings of up to 10.2%, the pattern Table 2 follows. The same literature warns what is *not* observable in an order table — product expectations, seller communication, returns handling — and any ceiling reported here must be read against that omission.

## 2.2 Model families for tabular transactional data

Regularised linear and logistic models are the natural baseline for structured data of this size: cheap, interpretable and, as Section 7 shows, revealing about distribution shift. Within the tree family the literature knows three ways to improve on one tree: a random forest bags decorrelated trees to cut variance [5]; boosting fits trees to residuals with shrinkage to cut bias [15]. Across the large empirical comparisons — ten algorithms on eleven problems [6], 179 classifiers on 121 datasets [13], 45 tabular benchmarks against deep architectures [18, 44] and 41 classifiers on business credit data [29] — boosted and bagged tree ensembles are the consistent top tier, boosting usually ahead by a small margin. In e-commerce and logistics the same family is the working choice: quantile regression forests for delivery times at JD.com [41], tree models for late supplier deliveries [1], and random forests or boosted trees in the published studies on this very dataset [14, 17, 49], none of which controls leakage by prediction moment — the gap Section 3 closes.

The boosted learner used here is `HistGradientBoosting`: LightGBM's histogram-binned design [26] implemented inside `scikit-learn` [42], so it stays within the brief's toolset rather than adding XGBoost [8]; it handles missing values natively, which the cold-start track-record features need; and it fits an order of magnitude faster than a comparable forest. Heterogeneous ensembles follow Wolpert [50]. The literature supplies the ranking; the ladder of Section 6 supplies what it cannot — whether, on this data, each rung beats the one below it.

## 2.3 Feature engineering on a relational order model

Deriving features from nine tables raises a problem a flat file does not: three of them are one-to-many against orders, so a naive join multiplies rows and reweights every aggregate towards multi-item orders, and each child relation must be aggregated to the analysis grain before joining. The feature groups then follow the literature. Origin–destination structure and distance are the backbone of published delivery-time models, from JD.com's arrival forecasts [41] to last-mile travel-time predictors [30]; the quoted promise encodes what the platform's routing knew [9]; order size, value and freight are the order-level controls through which Deshpande and Pendem [11] identify the logistics–rating link, and demand peaks stress fulfilment. The past performance of the parties to a transaction is the standard predictor in supply-chain delay models [1], and an as-of aggregate of a key's earlier outcomes is the time-respecting form of target encoding [31], which must be fitted inside the cross-validation loop. The general vocabulary of counts, ratios and calendar encodings follows Kuhn and Johnson [28] and Heaton [19].

Leakage is the dominant risk. Kaufman et al. [25] formalise it as information about the target unavailable at prediction time and identify the two forms that threaten this dataset, post-outcome features and improper partitioning; Kapoor and Narayanan [24] show how routinely both survive peer review, across 294 affected papers in 17 fields. Ours are their *illegitimate feature* and *no clean separation* categories, and the framing makes leakage relative to a *prediction moment*: delivery outcomes are forbidden when forecasting delivery and admissible when predicting a review not yet written. On partitioning, Bergmeir and Benítez [2] show that standard $k$-fold cross-validation is invalid for temporally ordered data.

## 2.4 Evaluation under imbalance, drift and capacity

Both classification targets are heavily imbalanced. Saito and Rehmsmeier [40] show that precision–recall curves are more informative than ROC curves under strong imbalance, because ROC-AUC is insensitive to a large true-negative mass; PR-AUC is reported here against its no-skill floor, with raw accuracy as a diagnostic beside the majority-class rule. The obvious correction is one the project tests rather than applies: van den Goorbergh et al. [47] find that under-sampling, over-sampling and SMOTE [7] do not improve discrimination and badly miscalibrate the resulting probabilities, and recommend adjusting the decision threshold on uncorrected data instead (Section 7.3 reports the test). Calibrated probabilities are then a requirement, because a cost-sensitive decision multiplies them by a cost [12], and calibration is reported, not assumed [33, 37].

Regression errors are judged against a naïve benchmark [22], decisive here because delivery performance improves dramatically across the window: under such a level shift $R^2$ against a baseline fitted on the past turns negative even for a useful model, whereas mean absolute error stays interpretable; this is concept drift [16], whose recommendation that non-stationary models be refitted with respect to time shapes the deployment plan. A ranking metric summarises the whole list, but a team contacts only the $K$ riskiest orders it can reach in a day, so the quantity that describes its day is the precision and recall of that top-$K$ — the targeting view of Provost and Fawcett [38], and the top-decile lift that profit-driven evaluation uses in place of accuracy [29, 48]. Whether an intervention changes the outcome is a causal question observational data cannot answer; the standard instrument is a randomised controlled experiment [27], which Section 9.4 designs. The deployment follows the practice literature: a model card stating what is served and where

it fails [32], and an explicit account of the version and data dependencies that quietly break production systems [43]. All modelling uses `scikit-learn` pipelines [36], keeping preprocessing inside each fold.

### 2.5 The gap addressed

The service-quality literature converges on fulfilment as the dominant observable driver of e-commerce satisfaction but measures it one way while the marketplace measures another; the tabular machine-learning literature converges on tree ensembles, and the e-commerce applications that use them rarely say *when* the model would run; and the methodological literature is emphatic that on imbalanced, temporally ordered data the choice of split, metric and imbalance handling decides whether a number means anything. The contribution here is narrow and concrete: make the prediction moment a first-class design decision, measure its cost, and evaluate at the capacity the stakeholder has.

## 3 Dataset and Preprocessing

### 3.1 Nine tables, one row per order

The dataset [34] comprises nine related CSV tables, roughly 1.45 million rows; `orders` is the spine, at 99,441 rows, one per order, and the other eight attach to it directly or through `order_items` (Figure 1; Table 12 in Appendix B is the full data dictionary). Three tables are **one-to-many** against orders, and joining them naively is the first trap the data sets: a direct orders–items–payments merge yields 118,437 rows rather than 99,441, a 19% inflation that silently reweights every mean towards large baskets. Each child table is therefore aggregated to one row per order — counts, sums and the attributes of the most expensive item — and the grain is asserted after the join.

The second trap is identity. `customer_id` is regenerated for every order, so it is an order key, not a person: 99,441 distinct values map to only **96,096** `customer_unique_id` values. Grouping on the wrong one reports zero repeat buyers; the right one finds 2,997 people (3.1%) who ordered again. `customer_unique_id` is carried throughout and the split is made on it.

Missingness is **structural rather than random**: 97.0% of orders reach status *delivered*, and the 2.98% with no delivery timestamp are orders whose delivery has not happened, so their delivery targets are undefined and excluded rather than imputed. Elsewhere the data is remarkably complete: 99.2% of orders carry a review and product attributes are missing for under 2% of products. The remaining issues — reused `review_id` values and orders with several reviews (keyed on `order_id`, earliest kept), free-text review fields too sparse for text features, a million geolocation rows collapsed to a median coordinate per postcode prefix, products without a category — are recorded with their evidence as `CLEANING_DECISIONS` in `src/data_load.py` and tabulated in Table 10 (Appendix B). Numeric imputation (median) happens inside each model pipeline, never on the frame.

### 3.2 Analysis window and the three-way split

The raw file spans September 2016 to October 2018, but the first four months are a pilot contributing 326 orders against a 2018 run rate above 6,000 a month, and the final weeks are truncated mid-cycle (Figure 2), so the analysis is restricted to **2017-01-01 to**

**2018-08-31**, leaving **99,091** orders.

Each period of the split has exactly one job. **Train** (2017-01-01 to 2018-04-30, 73,008 orders) fits every model, with expanding-window cross-validation inside it tuning hyperparameters. **Validation** (2018-05-01 to 2018-06-30, 13,156) chooses between tuned models, sets every decision threshold and fits the margin-to-risk calibration. **Test** (2018-07-01 to 2018-08-31, 12,927) is scored once, at the end, for models already chosen, and is the held-out window the deployed application replays (Section 8). The split is chronological rather than stratified-random because the target drifts (Section 4) and a random split would train on the future; the positive rate of each period is reported instead — late orders 8.8%, 5.0% and 7.4% of the training, validation and test periods, low reviews 15.8%, 11.4% and 11.0%. It is additionally disjoint on the customer: a purely temporal cut can still put one person's orders on each side, so the **227** straddling buyers are removed from a clean test subset used in Section 7.3.

### 3.3 Leakage: one rule per prediction moment

Leakage here is not one rule but one per *prediction moment*, because what is known depends on when the model runs. *At checkout* nothing about fulfilment or the customer's opinion exists, so delivery dates and everything derived from them, the review score and its timestamps, `order_approved_at`, `shipping_limit_date` and `order_status` are forbidden. *At delivery* only the review and its timestamps remain forbidden: the delivery outcome is now history, and the brief itself lists "delivery performance" among the features for that problem. `assert_no_leakage(features, regime)` enforces the correct set before any model is fitted, is tested in both directions, and raises on an unrecognised regime name.

`order_estimated_delivery_date` is admissible at checkout and matters: the customer is shown that promise when the order is placed, so it precedes the outcome and encodes whatever Olist's own routing knew. It is a useful feature and — Section 7.1 — a poor prediction. The cost of getting the regime wrong is measurable: adding one forbidden column, the realised delivery time, lifts the late-delivery model's ROC-AUC from 0.589 to **1.000** (Figure 9). That is not a result; it is the target restated.

## 4 Exploratory Analysis Highlights

Four findings from the exploratory phase shaped everything that followed; the figures are in Appendix O and every number is regenerated by `src/report_stats.py`.

### 4.1 Growth, seasonality and drift

Monthly order volume rises roughly **eight-fold** across the window, from about 800 orders in January 2017 to 6,000–7,500 a month through 2018, peaking at **Black Friday, November 2017** with 7,544 orders, 1.5 times the median month (Figure 3a). Purchasing is weekday-heavy, peaks at 16:00 and is 22% weekend; lead time is right-skewed with a median of 10.2 days and a long tail (Figure 4).

The most consequential temporal finding is not seasonality but **drift**. Mean lead time falls from 16.9 days in February 2018 to 7.7 in August 2018 — Olist materially improved fulfilment over twenty months (Figure 3b) — and across the chronological split the mean falls by **5.34 days** between training and test. Under a level shift this size, $R^2$ against a baseline fitted on the past is negative even for a useful model, so mean absolute error leads the regression results [22], and periodic refitting is a deployment requirement rather

than an optimisation [16].

## 4.2 Lateness: volatile, with identifiable shocks

Lateness behaves differently: it is **volatile rather than trending**, swinging from 1.4% (June 2018) to 21.4% (March 2018). The Phase 1 report attributed the spikes to Black Friday and to the May 2018 truckers' strike; the evaluators pointed out that the figure did not support the second claim, and `src/lateness_anatomy.py` re-examined the series at three resolutions (Appendix H, Figure 23). Black Friday 2017 is a genuine spike (14.3%). The largest peaks, February and March 2018 (16.0% and 21.4%), are a network-wide **carrier-side slowdown**: transit from carrier handoff to doorstep lengthened from 8.7 to 13.2 days while sellers' handling time did not move, and it coincided with the platform *tightening* its promised window from 26.3 days in January to 22.0 in March. The truckers' strike of 21 May to 1 June 2018 [4, 45] is real but invisible at monthly resolution: orders caught in transit were 16.0% late against a 5.0% average over the preceding five weeks, and the platform then lengthened its promise from 19.5 to 36.8 days, which is why June 2018 is the quietest month in the series. None of these shocks is an attribute of an order, so nothing available at checkout can anticipate them — the ceiling Section 1.4 states in advance.

## 4.3 Geography, products and payment

The marketplace is geographically **lopsided**, and that asymmetry drives the delivery problem: **59.7%** of sellers sit in São Paulo state against **42.0%** of customers, only 35.7% of orders stay within the customer's own state, and the median customer–seller distance is **434 km** (Figure 5). Distance is the strongest checkout-time correlate of lead time ($r = 0.39$), ahead of the platform's own estimate ($r = 0.38$). The full correlation matrix (Figure 8) adds three facts the models inherit: promise and distance correlate 0.52, so the promise is largely a distance rule; freight tracks weight (0.64) rather than price; and review score correlates $-0.33$ with lead time and essentially zero with price or basket size — the delivery, not the purchase, drives the rating. Demand is long-tailed across 74 categories; the typical order is small (median item value R$86.90) but freight is not a rounding error (median R$17.17, 22% of item value), and 51.4% of orders are paid in instalments (Figure 6).

## 4.4 Reviews, and the relationship that dominates

Review scores are severely imbalanced — **77.1%** are four or five stars — exactly the imbalance the brief warns about; the interesting structure is what produces the minority. A late delivery raises the probability of a 1–2 star review from **9.2%** to **54.0%**, a **5.9-fold** increase, monotone in how late the parcel is (Figure 7). This is the clearest relationship in the dataset, and it reproduces at order level the logistics–rating link Deshpande and Pendem [11] establish across 15 million orders. It also has a methodological consequence: a model predicting the review *at checkout* cannot see the variable that matters most.

## 4.5 From candidates to the two problems

Three candidate problems were probed on the chronological, customer-grouped split with untuned `scikit-learn` pipelines (Figure 10; Table 11 in Appendix B). *Delivery performance framed twice* became Problem 1: the regression reached MAE 3.89 days against 6.65 for a constant, while the direct late classifier was weak (PR-AUC 0.121 against a 0.074 floor)

and always predicting "on time" scored 0.926 accuracy while catching nothing. *Low review at delivery* became Problem 2, the strongest classifier found (PR-AUC 0.402 against a 0.110 floor); the same target predicted *at checkout* reached only 0.212, which made the prediction moment a design decision rather than a detail. *Repeat purchase within 30 days* was rejected: a 1.8% base rate, balanced accuracy 0.517, and right-censoring that leaves no labelled orders in the test window at the more natural 90-day horizon.

# 5 Feature Engineering

## 5.1 The 36 features, and why each is there

Phase 1's 25 features carry forward unchanged: 6 calendar (month, weekday, hour, weekend, business hours, the Black Friday window), 11 basket-and-money (`log_price`, `freight_ratio`, `log_weight`, volume, item and seller counts, instalments), 4 geographic (`log_distance`, `same_state`, customer state, seller region), 2 for the promise and 2 catalogue. The design carries no exact duplicates, and `purchase_year` is absent because the split is chronological, so a linear term on it would extrapolate into an unseen year. Phase 2 added **eight track-record features**, taking the checkout set to 33 and the at-delivery set to 36: a seller's mean lead time, late rate, low-review rate and order count; a route's mean lead time; a category's low-review rate; and the platform's own lead time and late rate. The at-delivery regime additionally sees three columns describing how delivery actually went.

Each group has a reason in the literature of Section 2 and a screen in the data. Geography and the promise are the backbone of delivery-time models [9, 30, 41]; basket, money and calendar are the order-level controls of the logistics–rating studies [11]; track records are the standard predictor in supply-chain delay models [1] in the time-respecting form of target encoding [31]. The empirical screen was the correlation matrix of Figure 8: lead time correlates 0.39 with distance, 0.38 with the promised window, 0.17 with freight and 0.07 with weight; promise and distance correlate 0.52 with each other, so both are kept and the trees resolve the overlap. Table 22 in Appendix K lists all 36 features with definition, rationale and the moment each becomes knowable.

## 5.2 Track records, and the trap inside them

A seller's history is the strongest thing a marketplace knows about an order at checkout, and the easiest place here to leak, because of a distinction that is easy to miss: *past* means the earlier order had **closed** before this one was placed — not merely that it was placed earlier. A parcel bought last week that has not yet arrived has no lead time yet, so ordering the history by *purchase* time would let every order average in outcomes that had not happened. That is Kaufman et al.'s *illegitimate feature* [25] in a convincing disguise, and it would be invisible in the metrics because it would simply look like a better model. Every aggregate is therefore an as-of join against the moment the earlier outcome became observable — `order_delivered_customer_date` for delivery, `review_creation_date` for reviews — with a strict boundary.

Two further guards. A mean over one previous order is noise rather than a rate, so below a floor of five prior orders the value is undefined and a fallback chain takes over (seller → route → seller state → platform), after which 0.95% of orders lack any history value; and cold start is information rather than error, since `hist_seller_n` carries how many past orders the estimate rests on. The platform features use a **28-day trailing window** rather than an expanding mean, because lead time halves across the analysis

window, and their range, 5.2 to 17.1 days, is a direct measure of the drift the models must survive. Because a leak here would be invisible in the metrics, the construction is checked rather than reviewed: the validation suite rebuilds `hist_seller_lead_days` from first principles for 200 random orders and requires exact agreement, and a second check confirms the platform tracker *lags* rather than anticipates (Appendix A).

# 6 Modelling Methodology

## 6.1 Model family budget and pipelines

The brief allows two to three families and asks for depth rather than breadth. We use three, reused across both task types, each arranged as a *complexity ladder* whose rung 0 is the family's simplest variant — the only way "the added complexity was worth it" becomes a finding rather than an assumption. **A — Linear** runs `LinearRegression` / `LogisticRegression` at rung 0 and `Ridge` / $\ell_2$-penalised logistic at rung 1. **B — Tree-based** runs `DecisionTree` → `RandomForest` → `HistGradientBoosting`, the three steps Section 2 argued from the literature. **C — Ensemble** is soft `Voting` and `Stacking` assembled from the winners of A and B; its members are complete pipelines rather than bare estimators, because the two families need different encodings and one shared preprocessor would force a single choice on both. The full specification, the search spaces and the selected values of every tuned model are in Appendix D.

Every model is a `scikit-learn` `Pipeline`: median imputation, one-hot encoding of the four categoricals with rare levels folded into one bucket, then — for the linear family only — standardisation, since trees are invariant to monotone rescaling and a shared penalty is not [36]. The linear family additionally drops one level per categorical, a correction rather than a preference: Phase 1 reported that ordinary least squares was unusable here, "rank-deficient, so its coefficients explode", and re-running it four ways (Appendix E) shows the predictions were always fine and only the coefficient vector was not; with full rank restored, Ridge and OLS agree to three decimals. Regularisation buys numerical stability here, not accuracy.

## 6.2 Cross-validation, and how a model gets chosen

Delivery performance drifts hard — mean lead time falls **5.34 days** between training and test — so shuffled $k$-fold lets a model learn from the future, the invalidity Bergmeir and Benítez [2] formalise. Hyperparameters are tuned by `TimeSeriesSplit` with five expanding windows inside the training period, searched with `RandomizedSearchCV` [3] over 30 draws per model (six for the costly random forests), scoring `neg_mean_absolute_error` for regression and `average_precision` for classification.

*Selecting* between tuned models is a separate question, and both obvious answers fail here for reasons that are properties of the data. Averaging cross-validated scores fails because the folds differ enormously in difficulty — lead time runs 12–17 days across the 2017 folds against 8.3 in the test window — so every model's CV error sits about a day above its forward error and the *ordering* does not survive: cross-validated MAE ranks Ridge above the boosted trees, while every later window and the test period rank them the other way round (Figure 11b). Using the validation period alone fails too: two months is short, and these two pull in opposite directions, the late rate being 8.2% in May 2018 and **1.4%** in June, the lowest in the series.

Each candidate is therefore scored on all six out-of-sample windows — the five expanding folds and the validation period — **ranked within each**, and the winner is the best mean rank over the **three most recent**. Ranks rather than scores, because ranks are the only thing comparable across windows of different difficulty and different base rates; three windows because that covers roughly February to June 2018, recent enough to resemble deployment and wide enough that neither of the validation period's two odd months decides alone. All six windows are reported for every candidate, with their fold-to-fold spread, in Appendix G; a shuffled, customer-disjoint `StratifiedGroupKFold` is computed alongside as a contrast exhibit and never used to choose anything. The selected models are *not* refitted on train plus validation before the test window is scored: that would decouple the shipped artifact from the threshold and calibration fitted against it, and what is reported here is exactly what is deployed.

### 6.3 Imbalance, metrics aligned to the action, and reproducibility

Both targets are imbalanced — 7.4% late and 11.0% low review in the test period — so accuracy alone is meaningless and every results row carries `majority_accuracy` beside `accuracy`, with PR-AUC reported against its no-skill floor [40]. Following van den Goorbergh et al. [47] we fit on the observed prior and move the *decision threshold* instead, tuned on the validation period by maximising F1, and Section 7.3 tests the alternative rather than assuming the result; no resampling is used anywhere [7]. Calibrated probabilities are a requirement rather than a nicety, because Section 9.2 multiplies them by a cost [12]; they are reported, not assumed [33].

A second alignment follows from Table 2. PR-AUC summarises the whole ranking and is the right criterion for *choosing* between models, but neither team acts on the whole ranking: customer experience contacts the $K$ riskiest deliveries its agents can reach in a day, and operations reviews the top of the late list. Section 7.4 therefore reports Precision@$K$ and Recall@$K$ for both flags [38, 48], with $K$ as a share of the day's volume so that the same table holds on a quiet Tuesday and on Black Friday, and Section 9.2 chooses the operating point as a capacity rather than as a threshold.

`SEED = 42` throughout. `src/train_models.py` fits everything and writes the models, metadata and every result table; `src/phase3_stats.py` distils those into `report/stats_phase3.txt`, and no number enters this report that it cannot regenerate; every figure is drawn from the same tables. Artifacts record the `scikit-learn` version that wrote them and the loader checks it; `tests/test_leakage.py` runs **60** validation checks, including one that ties the deployed artifacts to the metadata and to this text (Appendix A).

## 7 Results and Evaluation

Everything below is the untouched test window, 2018-07-01 to 2018-08-31, scored once for models already chosen. Cross-validated and validation columns sit beside it because the disagreement between them is part of the result; Appendix J adds the in-sample training fit of the deployed models, and Section 7.6 carries the error analysis by month and region. Figures are in Appendix O.

**Table 3:** Lead-time regression: two reference points that are not models, then each family's complexity ladder. CV MAE is the mean over the five expanding folds with its fold-to-fold standard deviation. **Bold** is the selected and deployed model. Figure 11.

| Model | Family | CV MAE ± sd | Val MAE | Test MAE | RMSE | $R^2$ |
|---|---|---|---|---|---|---|
| Mean of the training period | baseline | — | — | 6.645 | 7.572 | −0.990 |
| Olist's own delivery promise | baseline | — | — | 9.788 | 12.155 | −4.130 |
| LinearRegression *(rung 0)* | A linear | 5.242 ± 1.00 | 4.965 | 4.181 | 5.639 | −0.104 |
| Ridge *(rung 1)* | A linear | 5.201 ± 0.95 | 5.164 | 4.309 | 5.775 | −0.158 |
| DecisionTree *(rung 0)* | B tree | 5.539 ± 0.91 | 5.927 | 6.116 | 8.664 | −1.606 |
| RandomForest *(rung 1)* | B tree | 5.457 ± 0.90 | 5.513 | 4.956 | 6.169 | −0.321 |
| HistGradientBoosting *(rung 2)* | B tree | 5.495 ± 0.81 | 4.802 | 4.141 | 5.517 | −0.057 |
| **VotingRegressor (A+B)** | C ensemble | 5.315 ± 0.87 | **4.784** | **3.898** | **5.342** | +0.009 |
| StackingRegressor (A+B) | C ensemble | 5.443 ± 0.84 | 4.792 | 4.121 | 5.430 | −0.024 |

## 7.1 Problem 1A — lead time

The selected model puts mean absolute error at **3.90 days**: **41.3% below** a constant fitted on the training period and **60.2% below the platform's own promised date**. That second comparison is the one operations cares about, and its direction is not anomalous — a promised date is a commercial decision, padded because a broken promise costs returns and retention [9]. It is an excellent feature and a poor forecast.

Three things matter more than the winner. **Regularisation does not pay:** Ridge is *worse* than plain least squares on validation (5.164 against 4.965) and on test (4.309 against 4.181), with a tuned $\alpha$ of 660 that cross-validation preferred and no later window did — within family A, rung 1 fails to beat rung 0. **Inside family B complexity does pay**, monotonically on validation: 5.927 → 5.513 → 4.802. And **only the ensemble has a positive** $R^2$; every other model is negative, the signature of a level shift rather than of a useless model, and the reason MAE leads throughout [22]. The blend beats both its members (4.181 and 4.141 against 3.898) by a specific mechanism: a tree cannot predict outside the range of the targets it was fitted on, and test-window lead times are systematically shorter than anything common in training, whereas a linear model extrapolates the level (Figure 12). Averaging is one member covering the other's exact failure, which is why the weights are fixed at 1:2 in the tree's favour rather than tuned.

## 7.2 Problem 1B — late against the promise

**Table 4:** The late-order flag, best of each route; the full seven-candidate table is in `report/results/clf_p1b.csv`. No-skill PR-AUC floor 0.074. Figure 14.

| Route | Model | ROC-AUC | PR-AUC | lift | bal. acc | prec. | recall | F1 |
|---|---|---|---|---|---|---|---|---|
| direct | best of five (HistGB) | 0.584 | 0.092 | 1.24× | 0.524 | 0.088 | 0.279 | 0.134 |
| derived | predicted > promised, unfitted | 0.569 | 0.109 | 1.47× | 0.569 | 0.274 | 0.175 | 0.214 |
| **derived** | **regression margin, calibrated** | **0.782** | **0.214** | **2.89×** | **0.705** | 0.137 | **0.826** | **0.235** |

The deliverable is the derived route, and that follows from the framing fixed in Section 1 rather than from the numbers: Problem 1 is the brief's dual framing, so the flag *is* the regression's own answer compared against the promised date. The five direct classifiers answer a different question — does the flag need a model of its own? It does not. The derived route reaches **2.89×** the no-skill floor where the best direct classifier reaches 1.24×, and it costs no additional model: the margin $\hat{y}$ − promised is mapped to a probability by a single-parameter logistic [37] fitted on validation. Even the zero-parameter rule — flag

whenever the model expects a miss — beats every fitted classifier.

**The selection protocol got this one wrong, and that is worth reporting.** Applied to these seven candidates the rolling-origin rule of Section 6.2 ranks a direct classifier first. On every in-training window the candidates sit within noise of one another — in the fifth fold they span 0.206 to 0.362, with the derived route mid-pack — so the rule separates them on noise; on the forward window the gap opens to a factor of 2.3. The later training folds and the validation period are dominated by the February–March 2018 carrier slowdown and by the week the strike caught in transit (Section 4.2); a classifier can fit a regime, whereas the derived route encodes a mechanism, and mechanisms transfer. We report the disagreement rather than presenting the rule as having chosen this, and note that confirming the preference would need a further forward window we do not have.

## 7.3 Problem 2 — low-review risk

**Table 5:** Low-review classification at delivery. CV is the mean PR-AUC over the five expanding folds with its fold-to-fold standard deviation (Figure 16; every fold in Table 17). Accuracy is printed beside the majority-class rule that scores 0.890 while catching nothing. No-skill PR-AUC floor 0.110. **Bold** is the selected and deployed model. Figure 15.

| Model | Fam. | CV ± sd | Val ROC | PR-AUC | lift | bal. acc | prec. | recall | F1 | Brier |
|---|---|---|---|---|---|---|---|---|---|---|
| LogisticRegression *(r0)* | A | 0.465 ± .095 | 0.320 | 0.659 | 0.291 | 2.63× | 0.636 | 0.328 | 0.365 | 0.345 | 0.090 |
| LogisticRegression ($\ell_2$) *(r1)* | A | 0.466 ± .095 | 0.320 | 0.659 | 0.291 | 2.63× | 0.636 | 0.317 | 0.371 | 0.342 | 0.090 |
| DecisionTree *(r0)* | B | 0.543 ± .092 | 0.389 | 0.722 | 0.396 | 3.59× | 0.677 | 0.562 | 0.391 | 0.461 | 0.080 |
| RandomForest *(r1)* | B | 0.568 ± .089 | 0.403 | 0.744 | 0.405 | 3.67× | 0.680 | 0.576 | 0.396 | 0.470 | 0.078 |
| HistGradientBoosting *(r2)* | B | 0.568 ± .088 | 0.407 | 0.749 | 0.415 | 3.76× | 0.680 | 0.579 | 0.396 | 0.471 | 0.078 |
| VotingClassifier (A+B) | C | 0.561 ± .084 | 0.404 | 0.747 | 0.407 | 3.69× | 0.683 | 0.477 | 0.422 | 0.448 | 0.079 |
| **StackingClassifier (A+B)** | C | 0.568 ± .087 | **0.409** | 0.749 | 0.414 | 3.75× | 0.681 | 0.575 | 0.398 | 0.470 | 0.078 |

The selected model reaches **ROC-AUC 0.749**, **balanced accuracy 0.681** and **PR-AUC 0.414 — 3.75× the no-skill floor**. Its raw accuracy is 0.901 against a majority-class rule scoring 0.890, which is exactly why accuracy is printed but never led with. The tree family beats the linear one by 1.43× on PR-AUC — but a *single depth-limited decision tree* already reaches 0.396 of the eventual 0.414, so 96% of the achievable signal comes from one interpretable tree. And the model does not memorise buyers: on the 12,617 test orders from customers never seen in training it scores PR-AUC **0.415** against 0.414 overall.

**What the fold-to-fold variation says.** The Phase 2 evaluation asked for the spread behind each cross-validated number; Figure 16 draws every candidate on every window and Table 17 lists them. The spread is large — a standard deviation of about 0.09 around every mean — and almost entirely a property of the *windows*: the mean over candidates runs from 0.376 on the first fold to 0.630 on the fourth, while within any one window the candidates spread by a median of 0.045. The three tree-based rungs are inseparable on this evidence (CV means of 0.568 apiece; the gap between the best two is under 0.0001), which is the empirical case for the rank-based rule of Section 6.2 and for reading the forward columns when the difference is this small. The linear family is separated from the trees on every single window, so that conclusion is robust to the spread.

**Class weighting: the ablation Phase 1 promised.** On the best tree model, weighting the loss moves ROC-AUC from 0.749 to 0.747 and PR-AUC from 0.415 to 0.411 — no

gain — while the Brier score degrades from **0.0778 to 0.1331**, a 71% deterioration, and mean predicted risk inflates from 0.119 to **0.337** against an actual rate of 0.110. The weighted model overstates every customer's risk threefold (Figure 17). We therefore fit on the observed prior and move the threshold instead: tuned on validation by maximising F1 it lands at 0.19 rather than 0.50, which on the test window raises F1 from 0.414 to **0.470** and recall from 0.297 to 0.398 at a precision cost of 0.686 to 0.575. Calibration holds where the volume is: in the two bins containing 11,892 of the 12,855 test orders the gap between predicted risk and observed rate is under 0.013.

## 7.4 Capacity-constrained evaluation: Precision@$K$ and Recall@$K$

**Table 6:** What a ranked list delivers at a fixed capacity, on the test window. $K$ is the share of orders acted on; the daily rows cut the list per purchase day, which is how a team actually works it. Break-even precision is $c_{\text{action}}/c_{\text{miss}}$ from Section 9.2. Figure 22.

| | **Review risk, at delivery** | | | **Late flag, at checkout: derived / direct** | | |
|---|---|---|---|---|---|---|
| **Capacity $K$** | **precision** | **recall** | **lift** | **precision** | **recall** | **lift** |
| top 1% | 0.705 | 0.064 | 6.4× | 0.270 / 0.111 | 0.036 / 0.015 | 3.6 / 1.5× |
| top 2% | 0.700 | 0.127 | 6.3× | 0.273 / 0.126 | 0.074 / 0.034 | 3.7 / 1.7× |
| top 5% | 0.678 | 0.307 | 6.1× | 0.274 / 0.104 | 0.185 / 0.071 | 3.7 / 1.4× |
| top 10% | 0.475 | 0.431 | 4.3× | 0.258 / 0.093 | 0.348 / 0.126 | 3.5 / 1.3× |
| top 15% | 0.369 | 0.502 | 3.3× | 0.234 / 0.089 | 0.473 / 0.181 | 3.2 / 1.2× |
| top 20% | 0.300 | 0.544 | 2.7× | 0.211 / 0.089 | 0.568 / 0.241 | 2.8 / 1.2× |
| top 30% | 0.236 | 0.641 | 2.1× | 0.175 / 0.086 | 0.707 / 0.349 | 2.4 / 1.2× |
| 10 a day (4.8%) | 0.572 | 0.248 | 5.2× | — | — | — |
| 20 a day (9.5%) | 0.440 | 0.378 | 4.0× | — | — | — |
| break-even precision | 0.125 (R\$5 : R\$40) | | | 0.300 (R\$12 : R\$40) | | |

Table 6 reads the same test predictions as Tables 4 and 5, but as a team with limited capacity would. Three things follow. **Capacity, not the threshold, is the binding constraint for the review flag.** At a capacity of 5% of deliveries — about ten contacts a day on this volume of 207 delivered orders a day — two contacts in three reach a customer who would in fact have left a 1–2 star review, 6.1 times the rate of contacting at random, and those contacts cover 30.7% of all such reviews. Precision stays above the 0.125 break-even at every capacity tried, so the policy is not a threshold but an instruction: work the day's list from the top until the agents run out; the F1-tuned and cost-optimal thresholds are points on this curve (Figure 22a). At ten contacts a day precision is 0.572 and recall 0.248, a little below the pooled 5% row because days differ in how many risky orders they hold. **Not every hit can still be reached**: for a late parcel the survey goes out once the promise passes and 87% of its low reviews precede delivery, so of the 0.572, 0.352 have not yet written and 0.220 already have, against 0.093 and 0.017 at random — still nearly three times break-even (Appendix M). **The derived late flag dominates the direct classifier at every capacity**, by a factor of two to nearly three on precision. **And it never clears the break-even for an expensive action**: its precision peaks at 0.274 and falls as $K$ grows, under the 0.300 a R\$12 expedite would need, so Table 2 confines expediting to a manual review of the very top of the list.

### 7.5 Does the third family earn its place, and what is deployed

Family C is selected on both problems and the honest answer differs. On **lead time** it earns its place: the Voting blend improves on the better of its members by 0.24 days of MAE (5.9%) and is the only model in Table 3 with a positive $R^2$, for the extrapolation reason above. On **review risk** it does not: Stacking wins the rank vote by a nose (mean rank 1.67 against 2.00, ahead on three of the six windows), then matches the boosted trees to three decimals on ROC-AUC and comes in 0.001 *below* them on PR-AUC — and the ensemble was *flattered*, because stacking's inner folds must partition the training set and so cannot be an expanding window (Appendix D). Being flattered and still failing to separate is a safer conclusion than the reverse.

**What is deployed.** The Phase 2 report named the deployed review model inconsistently; the artifacts settle it. The served models are the selection rule's winners, read back from the pickles: the **VotingRegressor** (linear + histogram-boosted, weights 1:2) for lead time, the derived route with its one-parameter calibration for the late flag, and the **StackingClassifier** — the tuned $\ell_2$ logistic regression and the tuned `HistGradientBoosting` classifier under a logistic meta-learner that weights the tree member 13.8 times the linear one — for review risk (Table 7). The single boosted model would lose nothing measurable; it is not what is deployed, because the selection rule was fixed before any test score was seen and switching afterwards would be the selection-on-test practice this report rules out. The practical cost is nil — the stacked artifact is 321 KB — and the validation suite asserts that the artifact, the metadata, the hyperparameter table and this text name the same estimator.

### 7.6 Error analysis: where the models fail

Across geography the review model is remarkably stable: ROC-AUC runs 0.744 to 0.778 across the five Brazilian regions, with lift over each region's own floor between 3.6× and 5.0× (Figure 21b). The regression is not: MAE runs from 3.42 days in the Southeast, where most orders are, to **6.84 days in the North**, and the bias is positive everywhere — +1.15 days in the Southeast rising to +4.77 in the North (Figure 21a), and the bias stays positive in both test months (Figure 13). The model *over-predicts lead time everywhere*, most severely where deliveries are longest, which is what a model fitted on a slower past does when fulfilment has improved underneath it, and the strongest argument for refitting on a rolling basis. Table 21 in Appendix J separates that drift from over-fitting: the blend's in-sample MAE (4.72) sits level with validation (4.78) and above test (3.90), and the stacked review model shows the modest in-sample optimism of a tree ensemble (ROC-AUC 0.836 in-sample against 0.762 and 0.749 on the periods it never saw) while validation and test agree with each other.

## 8 Model Deployment and Application

The proof of concept is a Streamlit application deployed on Streamlit Community Cloud at `https://team10-it5006-olist.streamlit.app`, built from the `deployment/` folder of the repository. It serves all three deployed models, and because the Olist extract ends in 2018 it does what the brief anticipates: it predicts on held-out historical records that stand in for new orders, and it can replay the test window a day at a time.

## 8.1 Architecture

**Table 7:** The deployed artifacts, as pickled. Thresholds and the margin-to-risk mapping were fitted on the validation period; nothing was refitted for deployment, so the served models are exactly the ones Section 7 evaluates.

| Problem | Estimator served | Artifact | Decision rule |
|---|---|---|---|
| 1A lead time | `VotingRegressor`: LinearRegression + HistGradientBoosting, weights 1:2 | 381 KB | re-quote with the estimate |
| 1B late flag | logistic map of (predicted − promised days) to a probability | 1 KB | flag at $p_{\text{late}} \geq 0.11$ |
| 2 low review | `StackingClassifier`: $\ell_2$ LogisticRegression + HistGradientBoosting, logistic meta-learner | 321 KB | flag at $p_{\text{low}} \geq 0.19$; rank for the top-$K$ |

Nothing in the application reimplements the training-time feature engineering. A form submission or an uploaded row is turned back into an order frame and pushed through the same `src/features.py` builders the models were fitted on, which is what stops a feature being computed one way in training and another way in production [43]. The one thing a form cannot supply is a seller's track record, so `models/history_lookup.csv` carries those aggregates at each level of the same fallback chain the training code uses. The artifacts total under 1 MB and record the `scikit-learn` version that wrote them; `requirements.txt` pins that version, the loader raises a visible warning if it ever drifts, and the app runs on Python 3.12, the interpreter the pinned numpy publishes wheels for. Models are loaded lazily, so a visitor who only browses the exploratory tabs never pays for unpickling one.

## 8.2 What the application does

The application has eight tabs (Figures 25 to 27). Five are the exploratory analysis of Section 4 with sidebar filters. **Predict** scores a single order entered as it would look at checkout and returns the expected lead time with its typical error, the risk of missing the promised date against the tuned threshold, and, conditional on that forecast, the low-review risk; it also scores an uploaded CSV of up to 5,000 orders and builds the **triage list**: rows ranked by review risk with the top-$K$ marked for a capacity the user enters, and the precision and recall that capacity achieved on the held-out window beside it. **Replay a day** is new in this phase (Figure 26): the user picks any purchase day in the test window and a daily capacity, sees what the deployed models predicted for every order placed that day, ranked the way the customer-experience team would work them, and a toggle reveals what actually happened — which orders left a 1–2 star review, how long delivery really took — with the precision of the top-$K$ on that day and on every other day of the window. For 16 July 2018 at ten contacts, eight of the ten would have reached a would-be low reviewer. **Model card** states what is deployed as read from the artifacts (Table 7), the training protocol, every searched model's selected hyperparameters, the fold-to-fold variation of every candidate, the Precision@$K$ table, the situations in which the models should not be trusted [32], and a calculator for the pilot of Section 9.4. Every number it shows is read from the result tables this report quotes; the application recomputes nothing.

## 8.3 Robustness, reachability and reproducibility

Invalid input produces a specific message rather than a traceback: a promised date before the purchase date, a non-positive price, more sellers than items, a malformed, empty or oversized upload, a missing column, an unknown category or state, a missing artifact, a version mismatch. The data loader asserts the row count of all nine source tables and stops the app if they change. There are no API endpoints and no credentials anywhere: the batch CSV contract, documented with an example in `deployment/README.md`, is the programmatic surface.

The Phase 1 dashboard became inaccessible when the GitHub account it was deployed from was suspended, which took the Community Cloud app down with it. The application now deploys from the team's current repository, and two things keep it reachable, both recorded in the README: Community Cloud puts an app without visitors for twelve hours to sleep behind a one-click wake-up button that any visitor can press, and the Python version must stay at 3.12 in the app's settings. A fresh clone reproduces everything, from the committed source tables to every table, figure and number in this document (Appendix A).

# 9 Business Recommendations and Impact

## 9.1 What the models rely on

Importance is measured by permutation on the held-out window rather than by impurity on the training set, which inflates high-cardinality features [46] — not hypothetical with 74 product categories (Figure 20). **Problem 2 has one dominant driver, and it is actionable.** How late the parcel was carries 0.206 of PR-AUC on its own, how long it took a further 0.074, the size of the order 0.045; nothing else clears 0.011, and the top two account for **76.9%** of total positive importance, which is why a single depth-limited tree captured 96% of the achievable signal. For the customer-experience team that is good news: the driver is something the business already measures. **Problem 1A's importances must be read with care.** No single feature carries much — the largest, `lead_payment_type`, costs 0.034 days of MAE — because permutation importance understates correlated predictors such as the promise, the route histories and the distance; this model has no single point of failure.

## 9.2 From a probability to a decision, and when not to act

A probability is not a decision. Turning one into the other needs a cost, and Olist publishes none — so what follows states its assumptions openly and treats the conclusions as shapes rather than numbers [12]. The arithmetic is one line: acting is worth it only where precision $> c_{\text{action}}/c_{\text{miss}}$ (Table 8).

**Table 8:** The decision layer under stated (not measured) costs. Break-even precision is what decides whether a flag is worth acting on at all.

| Flag | $c_{\text{action}}$ : $c_{\text{miss}}$ | break-even | best achieved | verdict |
|---|---|---|---|---|
| Low-review risk, at delivery | R\$5 : R\$40 | 0.125 | 0.37 at the optimum | **act** |
| Late-order flag, at checkout | R\$12 : R\$40 | 0.300 | 0.29 at any volume | do not expedite |

**The review flag pays, comfortably.** Contacting a customer who was never going to complain costs perhaps R\$5 of agent time; a 1–2 star review that could have been headed

off costs perhaps R$40 — standing in for the damage Deshpande and Pendem [11] trace through to seller sales. Break-even precision is 0.125 and the model reaches 0.369 at its expected-benefit optimum, flagging 1,927 of 12,855 delivered orders (15.0%) of which 712 do go on to leave a low review, for a net **R$1,466 per 1,000 delivered orders** (R$775 counting only customers who have not yet written, Appendix M; Figure 18); the margin is wide enough to survive a wrong cost estimate. Capacity changes how the optimum is used rather than where it is: a team that can work only 5% of deliveries still captures R$1,107 of those R$1,466 (Table 6), because the top of the list is where the precision is, so the list is simply worked top-down until the agents run out.

**The late-order flag does not pay — for an expensive action.** This is the more useful finding, and reporting discrimination alone would have missed it. Expediting freight costs perhaps R$12 against the same R$40 of downstream damage, so break-even precision is 0.300 — and the derived flag never reaches it: precision plateaus at **0.29** and falls as volume rises. A model with ROC-AUC 0.782 and 2.9× the no-skill floor is genuinely informative and still not worth paying R$12 an order to act on. That changes *which* action it justifies rather than making it useless: break-even falls with the cost of the intervention, and re-quoting the date and pre-warning the buyer cost almost nothing.

## 9.3 Recommendations by stakeholder

**Operations, at checkout** should work the four levers of Table 2 in order of cost. Every order gets the re-quoted date: the model's estimate is off by 3.90 days against the promise's 9.79, so the date shown at checkout can be tightened materially, which Cui et al. [9] show trades directly against returns and retention. Every order above the recall-oriented threshold of 0.11 gets a proactive notice, which covers five in six of those that will actually miss. Expedited freight is reserved for a manual review of the top one or two per cent of the late list, where precision is 0.27, and that policy is revisited only if expediting falls below about R$11 an order. The weekly late series is the monitoring instrument: a week in which transit lengthens while the promise does not is the February–March 2018 pattern again (Appendix H).

**Customer experience, at delivery** should receive each day's deliveries ranked by $p_{\text{low}}$ and work the list from the top. At ten contacts a day — 4.8% of deliveries — 57% of contacts reach a customer who would have left a 1–2 star review, 35% one who has not yet written it; twice the capacity reaches 38% of them at 44% precision (Table 6). The deployed application returns exactly this list. **Customer care, on the promised date,** takes the late parcels: most of their low reviews are written before delivery, so the customer is reached as the promise breaks, with an apology and a new date, and the survey is sent after arrival (Appendix M).

**The platform** should refit both models monthly and monitor the gap between predicted and realised lead time as the trigger: the drift of Section 4 is the dominant risk to both, and the over-prediction of Section 7.6 is what an unrefitted model looks like after four months.

## 9.4 Measuring what the business case assumes: the outreach pilot

Every reais figure above rests on one assumption the data cannot test: that contacting a customer who would have left a low review prevents it. Olist never contacted anyone, so the counterfactual is unobserved, and the evaluators rightly asked that it be tested rather

than assumed. The test is a randomised controlled experiment [27], and it can be sized from numbers this report already has (`src/pilot_design.py`; Table 23 and Figure 24).

*Design.* Each day the model ranks the day's deliveries and the team takes the top $K$. Within that top $K$, orders are randomised 1:1 — half contacted, half left alone — and the outcome is the 1–2 star rate in each arm. Randomising *inside* the list is what makes the comparison fair: both arms have the same predicted risk. The control arm's rate is already known: it is the model's precision at that daily $K$, 0.572 at ten contacts a day. *Break-even* is one line of arithmetic: a contact pays when the reduction in the low-review probability times the review's cost exceeds the contact's cost, so the required absolute reduction is $5/40 = 0.125$ and the required relative reduction $0.125/0.572 = 21.9\%$ — the contact must avert roughly one low review in five of those it reaches. *Size.* For a two-sided two-proportion test at $\alpha = 0.05$, detecting a one-in-four reduction (0.572 to 0.429) with 80% power needs 191 orders per arm, about **six weeks** at ten contacts a day (eight at 90% power); smaller effects take longer (Table 23). The primary outcome is the review rate; the secondary is the realised benefit per contact, $(\hat{p}_{\text{control}} - \hat{p}_{\text{treated}}) \times \text{R\$40} - \text{R\$5}$, which replaces the assumed benefit with a measured one. A second arm splits the late parcels the same way on their promised date, testing the apology and the later survey.

### 9.5 Impact, and what it would take

Under the stated costs the review-risk list is worth R\$775–1,466 per thousand delivered orders at the cost optimum, depending on whether customers who have already written count, before the pilot replaces the assumed benefit with a measured one. The operations levers cost nothing to apply and act on every order: a promise re-quoted at MAE 3.90 days instead of 9.79, and a pre-warning that reaches five in six of the orders that will miss. What it would take is modest: the three artifacts and a daily scoring job, which the application already runs on a held-out day; a monthly refit; and a six-week pilot before the customer-experience policy is scaled.

## 10 Limitations and Conclusion

**The models are fitted on a moving target.** Mean lead time falls 5.34 days between training and test, and a model fitted on the slower past is systematically off-level on the faster present — Section 7.6 measures that as a positive bias in every region. Hence $R^2$ against a test-period variance is negative even for a useful model, which is why MAE leads, and hence the models need refitting on a rolling basis rather than once [16]; calibration belongs in the same cycle, because a calibration fitted on one prior drifts as the prior does.

**The largest driver of lateness is not observable at prediction time.** The late rate swings between 1.4% and 21.4% month to month, and Section 4.2 traces the swings to a Black Friday surge, a carrier-side slowdown that coincided with a tightened promise, and a national strike absorbed into a longer promised window: shocks to carrier capacity or to the platform's own promising policy, none an attribute of an order. This, not the choice of algorithm, is the binding constraint on Problem 1B, and it is why the ceiling was stated in advance rather than a weak classifier presented as a disappointment.

**The benefit of outreach is assumed, not measured.** Section 9.2 values a contact at the review it prevents, and the data contain no contacts: nobody in this history was

reached before reviewing, so the effect of reaching them — and the possibility that an unsolicited contact lowers a rating — is unobserved. The precision and recall figures stand on their own; the reais figures do not, and should be read as the value of the policy *if* the pilot of Section 9.4 finds the contact averts at least one low review in five. Part of the assumption can be checked, and fails for late parcels: most of their low reviews are written before delivery, so the value is restated on the reachable share (Appendix M). Until the pilot runs, the recommendation is to pilot for six weeks, not to scale.

**What the data does not contain, and what rests on assumption.** The target of Problem 2 is a platform rating, with the self-selection Section 1 describes: a model of the rating is not a model of satisfaction, and the 1–2 star label cannot see the quietly dissatisfied customer who never reviews. The service-quality literature identifies product expectation gaps, seller communication and returns handling among the drivers of dissatisfaction [35, 39]; none appears in these nine tables, so the Problem 2 residual is unobserved cause rather than noise, and the free-text review columns are too sparse to recover it. Selection votes on three out-of-sample windows — the right trade under drift, but a small sample: Section 7.2 records a case where the rule preferred a model that does not transfer, and Section 7.3 shows a fold-to-fold spread too wide to separate the best candidates at all.

**Generalisation, and what these scores are not.** Everything here is Brazil, 2017–2018, on one marketplace whose geography — 59.7% of sellers in São Paulo state against 42.0% of customers, a median order travelling 434 km — the models' distance-driven structure reflects. The outputs are triage scores for internal prioritisation, not entitlements, and must not be used to penalise a seller, price a customer, or drive an automated decision about someone without review; the review-risk model is largely a restatement of lateness, so a seller penalised by it would often be penalised for a carrier's failure.

## Conclusion

Two problems, at two prediction moments, from one dataset and one feature pipeline, carried through to a deployed application. The regression reaches MAE 3.90 days — 41% below a constant fitted on the past and 60% below the platform's own promise — and its output, thresholded against that promise, makes a better late-order flag (2.89× the no-skill floor) than any classifier trained directly on the binary label (1.24×), so the brief's dual framing turns out to be the substantively better design. The satisfaction model reaches 3.75× its floor on a target where always predicting "satisfied" scores 89% accuracy and catches nothing, and read as a ranked list it turns ten contacts a day into an outreach queue of 57% precision — a value limited by capacity, not by the model's threshold.

The methodological findings travel further. Cross-validation inside a drifting training period reorders the candidates relative to the forward window, and its fold-to-fold spread is too wide to tell the best candidates apart — so selection belongs on several recent out-of-sample windows judged by rank, and even then it can fail. Weighting the loss for class imbalance leaves ranking unchanged and inflates predicted risk threefold, so the threshold is the right lever. And the most useful business results are negative: an informative flag that does not justify an expensive intervention, and a paper gain of R$775–1,466 per thousand orders that is worth a six-week experiment before it is worth anything.

# References


[1] George Baryannis, Samir Dani, and Grigoris Antoniou. Predicting supply chain risks using machine learning: The trade-off between performance and interpretability. *Future Generation Computer Systems*, 101:993–1004, 2019. doi: 10.1016/j.future.2019.07.059.

[2] Christoph Bergmeir and José M. Benítez. On the use of cross-validation for time series predictor evaluation. *Information Sciences*, 191:192–213, 2012. doi: 10.1016/j.ins.2011.12.028.

[3] James Bergstra and Yoshua Bengio. Random search for hyper-parameter optimization. *Journal of Machine Learning Research*, 13(10):281–305, 2012.

[4] Bloomberg News. Brazil truckers strike for fourth day despite fuel price cut. Bloomberg, 24 May 2018. `https://www.bloomberg.com/news/articles/2018-05-24/brazil-truckers-strike-for-fourth-day-despite-fuel-price-cut`, 2018. Accessed 9 October 2026.

[5] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001. doi: 10.1023/A:1010933404324.

[6] Rich Caruana and Alexandru Niculescu-Mizil. An empirical comparison of supervised learning algorithms. In *Proceedings of the 23rd International Conference on Machine Learning (ICML '06)*, pages 161–168. ACM, 2006. doi: 10.1145/1143844.1143865.

[7] Nitesh V. Chawla, Kevin W. Bowyer, Lawrence O. Hall, and W. Philip Kegelmeyer. SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16:321–357, 2002. doi: 10.1613/jair.953.

[8] Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, 2016. doi: 10.1145/2939672.2939785.

[9] Ruomeng Cui, Zhikun Lu, Tianshu Sun, and Joseph M. Golden. Sooner or later? promising delivery speed in online retail. *Manufacturing & Service Operations Management*, 26(1):233–251, 2024. doi: 10.1287/msom.2021.0174.

[10] Chrysanthos Dellarocas. The digitization of word of mouth: Promise and challenges of online feedback mechanisms. *Management Science*, 49(10):1407–1424, 2003. doi: 10.1287/mnsc.49.10.1407.17308.

[11] Vinayak Deshpande and Pradeep K. Pendem. Logistics performance, ratings, and its impact on customer purchasing behavior and sales in E-Commerce platforms. *Manufacturing & Service Operations Management*, 25(3):827–845, 2023. doi: 10.1287/msom.2021.1045.

[12] Charles Elkan. The foundations of cost-sensitive learning. In *Proceedings of the Seventeenth International Joint Conference on Artificial Intelligence (IJCAI-01)*, pages 973–978, Seattle, Washington, 2001.

[13] Manuel Fernández-Delgado, Eva Cernadas, Senén Barro, and Dinani Amorim. Do we need hundreds of classifiers to solve real world classification problems? *Journal of Machine Learning Research*, 15:3133–3181, 2014.

[14] Maha Fouad, Sherif Barakat, and Amira Rezk. Effective E-commerce based on predicting the level of consumer satisfaction. In *Green Sustainability: Towards Innovative Digital Transformation*, Lecture Notes in Networks and Systems, pages 261–278. Springer Nature Singapore, 2023. doi: 10.1007/978-981-99-4764-5_17.

[15] Jerome H. Friedman. Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, 29(5):1189–1232, 2001. doi: 10.1214/aos/1013203451.

[16] João Gama, Indrė Žliobaitė, Albert Bifet, Mykola Pechenizkiy, and Abdelhamid Bouchachia. A survey on concept drift adaptation. *ACM Computing Surveys*, 46(4), 2014. doi: 10.1145/2523813.

[17] Akula Venkata Satya Sai Gopinadh, S. V. S. N. Sarma, and Gudipudi Radhesyam. Customer satisfaction prediction in online goods delivery through interpretable predictive models and sentiment analysis. In *Proceedings of the 7th International Conference on Communications and Cyber Physical Engineering*, Lecture Notes in Electrical Engineering, pages 1015–1025. Springer Nature Singapore, 2025. doi: 10.1007/978-981-95-0269-1_114.

[18] Léo Grinsztajn, Edouard Oyallon, and Gaël Varoquaux. Why do tree-based models still outperform deep learning on typical tabular data? In *Advances in Neural Information Processing Systems 35 (NeurIPS 2022) Datasets and Benchmarks Track*, volume 35, pages 507–520, 2022.

[19] Jeff Heaton. An empirical analysis of feature engineering for predictive modeling. In *SoutheastCon 2016*, pages 1–6. IEEE, 2016. doi: 10.1109/SECON.2016.7506650.

[20] Nan Hu, Jie Zhang, and Paul A. Pavlou. Overcoming the J-shaped distribution of product reviews. *Communications of the ACM*, 52(10):144–147, 2009. doi: 10.1145/1562764.1562800.

[21] Nan Hu, Paul A. Pavlou, and Jie Zhang. On self-selection biases in online product reviews. *MIS Quarterly*, 41(2):449–471, 2017. doi: 10.25300/MISQ/2017/41.2.06.

[22] Rob J. Hyndman and Anne B. Koehler. Another look at measures of forecast accuracy. *International Journal of Forecasting*, 22(4):679–688, 2006. doi: 10.1016/j.ijforecast.2006.03.001.

[23] Shanthan Kandula, Srikumar Krishnamoorthy, and Debjit Roy. A prescriptive analytics framework for efficient E-commerce order delivery. *Decision Support Systems*, 147:113584, 2021. doi: 10.1016/j.dss.2021.113584.

[24] Sayash Kapoor and Arvind Narayanan. Leakage and the reproducibility crisis in machine-learning-based science. *Patterns*, 4(9):100804, 2023. doi: 10.1016/j.patter.2023.100804.

[25] Shachar Kaufman, Saharon Rosset, Claudia Perlich, and Ori Stitelman. Leakage in data mining: Formulation, detection, and avoidance. *ACM Transactions on Knowledge Discovery from Data*, 6(4), 2012. doi: 10.1145/2382577.2382579.

[26] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems 30 (NIPS 2017)*, pages 3146–3154, 2017.

[27] Ron Kohavi, Roger Longbotham, Dan Sommerfield, and Randal M. Henne. Controlled experiments on the web: Survey and practical guide. *Data Mining and Knowledge Discovery*, 18(1):140–181, 2009. doi: 10.1007/s10618-008-0114-1.

[28] Max Kuhn and Kjell Johnson. *Feature Engineering and Selection: A Practical Approach for Predictive Models*. Chapman and Hall/CRC, Boca Raton, 2019. doi: 10.1201/9781315108230.

[29] Stefan Lessmann, Bart Baesens, Hsin-Vonn Seow, and Lyn C. Thomas. Benchmarking state-of-the-art classification algorithms for credit scoring: An update of research. *European Journal of Operational Research*, 247(1):124–136, 2015. doi: 10.1016/j.ejor.2015.05.030.

[30] Sheng Liu, Long He, and Zuo-Jun Max Shen. On-time last-mile delivery: Order assignment with travel-time predictors. *Management Science*, 67(7):4095–4119, 2021. doi: 10.1287/mnsc.2020.3741.

[31] Daniele Micci-Barreca. A preprocessing scheme for high-cardinality categorical attributes in classification and prediction problems. *ACM SIGKDD Explorations Newsletter*, 3(1):27–32, 2001. doi: 10.1145/507533.507538.

[32] Margaret Mitchell, Simone Wu, Andrew Zaldivar, Parker Barnes, Lucy Vasserman, Ben Hutchinson, Elena Spitzer, Inioluwa Deborah Raji, and Timnit Gebru. Model cards for model reporting. In *Proceedings of the Conference on Fairness, Accountability, and Transparency (FAT* '19)*, pages 220–229, New York, NY, USA, 2019. ACM. doi: 10.1145/3287560.3287596.

[33] Alexandru Niculescu-Mizil and Rich Caruana. Predicting good probabilities with supervised learning. In *Proceedings of the 22nd International Conference on Machine Learning (ICML '05)*, pages 625–632. ACM, 2005. doi: 10.1145/1102351.1102430.

[34] Olist and Sionek, André. Brazilian E-Commerce public dataset by Olist. Kaggle, 2018. Distributed for IT5006 via Canvas; nine relational CSV tables covering 99,441 orders placed between 2016 and 2018.

[35] A. Parasuraman, Valarie A. Zeithaml, and Arvind Malhotra. E-S-QUAL: A multiple-item scale for assessing electronic service quality. *Journal of Service Research*, 7(3): 213–233, 2005. doi: 10.1177/1094670504271156.

[36] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[37] John C. Platt. Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. In Alexander J. Smola, Peter L. Bartlett, Bernhard Schölkopf, and Dale Schuurmans, editors, *Advances in Large Margin Classifiers*, pages 61–74. MIT Press, Cambridge, Massachusetts, 1999.

[38] Foster Provost and Tom Fawcett. *Data Science for Business: What You Need to Know about Data Mining and Data-Analytic Thinking*. O'Reilly Media, Sebastopol, California, 2013.

[39] Shashank Rao, Thomas J. Goldsby, Stanley E. Griffis, and Deepak Iyengar. Electronic logistics service quality (e-LSQ): Its impact on the customer's purchase satisfaction and retention. *Journal of Business Logistics*, 32(2):167–179, 2011. doi: 10.1111/j.2158-1592.2011.01014.x.

[40] Takaya Saito and Marc Rehmsmeier. The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLOS ONE*, 10(3):e0118432, 2015. doi: 10.1371/journal.pone.0118432.

[41] Nooshin Salari, Sheng Liu, and Zuo-Jun Max Shen. Real-time delivery time forecasting and promising in online retailing: When will your package arrive? *Manufacturing & Service Operations Management*, 24(3):1421–1436, 2022. doi: 10.1287/msom.2022.1081.

[42] scikit-learn developers. HistGradientBoostingClassifier — scikit-learn 1.5.2 documentation. `https://scikit-learn.org/1.5/modules/generated/sklearn.ensemble.HistGradientBoostingClassifier.html`, 2024. Histogram-based gradient boosting "inspired by LightGBM", with native support for missing values. Accessed 9 October 2026.

[43] D. Sculley, Gary Holt, Daniel Golovin, Eugene Davydov, Todd Phillips, Dietmar Ebner, Vinay Chaudhary, Michael Young, Jean-François Crespo, and Dan Dennison. Hidden technical debt in machine learning systems. In *Advances in Neural Information Processing Systems 28 (NIPS 2015)*, pages 2503–2511, 2015.

[44] Ravid Shwartz-Ziv and Amitai Armon. Tabular data: Deep learning is not all you need. *Information Fusion*, 81:84–90, 2022. doi: 10.1016/j.inffus.2021.11.011.

[45] Victor Hugo Silva. Correios suspendem entregas do Sedex por conta de paralisação de caminhoneiros. Tecnoblog, 23 May 2018. `https://tecnoblog.net/noticias/correios-suspendem-entregas-sedex/`, 2018. In Portuguese. Correios suspended its time-definite Sedex services and added five business days to standard Sedex and PAC lead times during the strike. Accessed 9 October 2026.

[46] Carolin Strobl, Anne-Laure Boulesteix, Achim Zeileis, and Torsten Hothorn. Bias in random forest variable importance measures: Illustrations, sources and a solution. *BMC Bioinformatics*, 8:25, 2007. doi: 10.1186/1471-2105-8-25.

[47] Ruben van den Goorbergh, Maarten van Smeden, Dirk Timmerman, and Ben Van Calster. The harm of class imbalance corrections for risk prediction models: Illustration and simulation using logistic regression. *Journal of the American Medical Informatics Association*, 29(9):1525–1534, 2022. doi: 10.1093/jamia/ocac093.

[48] Wouter Verbeke, Karel Dejaeger, David Martens, Joon Hur, and Bart Baesens. New insights into churn prediction in the telecommunication sector: A profit driven data mining approach. *European Journal of Operational Research*, 218(1):211–229, 2012. doi: 10.1016/j.ejor.2011.09.031.

[49] P. Wangkiat and C. Polprasert. Machine learning approach to predict E-commerce customer satisfaction score. In *2023 8th International Conference on Business and Industrial Research (ICBIR)*, pages 1176–1181. IEEE, 2023. doi: 10.1109/ICBIR57571.2023.10147542.

[50] David H. Wolpert. Stacked generalization. *Neural Networks*, 5(2):241–259, 1992. doi: 10.1016/S0893-6080(05)80023-1.

# A Reproducing this report

Table 9 is the run order. `SEED = 42` is set in every estimator, every randomised search and every subsample. Both requirements files pin exact versions on Python $\leq$ 3.12, the interpreter every reported number was produced on. Model artifacts record the `scikit-learn` version that wrote them and `modeling.load_model()` checks it on the way back in. The nine notebooks (01–04 exploratory, 05–09 modelling, 10 the additions of this phase) are committed with their outputs.

# B Data model, cleaning decisions and dictionary

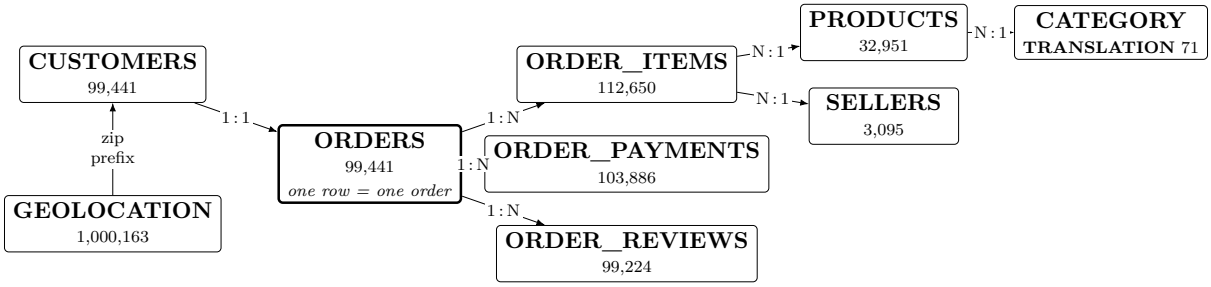



**Figure 1:** The nine source tables. `ORDERS` is the analysis grain. The three one-to-many children (items, payments and reviews) are each aggregated to one row per order *before* joining; merging them directly inflates the table by 19% and biases every average.

**Table 9:** Everything in this document is generated. The order below is the order to run it in. The fitting step dominates; the rest takes a few minutes.

| Command | What it produces |
|---|---|
| `pip install -r requirements-dev.txt` | The full analysis environment. `requirements.txt` alone is the deployment runtime |
| `python tests/test_leakage.py` | 60 validation checks: join grain, the history construction against a brute-force recomputation, the leakage guard in both directions, split disjointness, Phase 1 backward compatibility, the capacity and lateness tables against recomputation, and — added in this phase — that the pickled models, the metadata, the hyperparameter table, the README and this report name the same deployed estimators, and that the review-timing value recomputes from the per-order predictions |
| `python src/train_models.py` | Fits every model, scores the test window once, writes `models/` and every result table into `report/results/` |
| `python src/capacity_metrics.py` | Precision@$K$ and Recall@$K$ from the saved per-order test predictions (Section 7.4); nothing is refitted |
| `python src/lateness_anatomy.py` | The weekly, monthly and strike-cohort lateness tables of Appendix H |
| `python src/fit_diagnostics.py` | The three deployed models scored on the training, validation and test periods (Appendix J) |
| `python src/selection_tables.py` | Every candidate's score on every out-of-sample window with its fold-to-fold spread (Appendix G), and the selected hyperparameters of every searched model and of the deployed ensembles' members, read from the pickles (Appendix D) |
| `python src/pilot_design.py` | The sizing of the outreach pilot (Section 9.4, Appendix L) |
| `python src/review_timing.py` | When each review was written against the delivery, the share of the call list that can still be reached, the value restated on that share, and the per-order flags the application's replay shows (Appendix M) |
| `python src/figures_phase2.py` | Draws every modelling figure as vector PDF into `report/figures/` from those tables; the exploratory figures are drawn by notebooks 01–04 |
| `python src/phase3_stats.py` | Distils every number quoted in this report into `report/stats_phase3.txt` (it runs `phase2_stats.py` first, so the Phase 2 contract is a subset). Diff it against the manuscript |
| `python src/report_stats.py` | Reproduces every exploratory number of Section 4 into `report/stats.txt`, unchanged since Phase 1 — the check that the later feature additions are backward compatible |
| `streamlit run deployment/app.py` | The application of Section 8; `dashboard/app.py` is the Community Cloud entry point and runs the same file |

**Table 10:** Data quality audit and cleaning decisions, implemented as `CLEANING_DECISIONS` in `src/data_load.py`.

| Issue | Evidence | Decision |
|---|---|---|
| One-to-many joins | 112,650 items and 103,886 payments against 99,441 orders; a naive merge yields 118,437 rows | Aggregate each child table to one row per order, then join; assert the grain |
| Customer key | 99,441 `customer_id` values but 96,096 `customer_unique_id`; 2,997 people ordered more than once | Carry `customer_unique_id` and group on it for repeat behaviour and for splitting |
| Undelivered orders | 2.98% of orders have no delivery timestamp; status is *delivered* for 97.0% | Delivery targets computed only for delivered orders; the rest are `NaN`, never imputed |
| Review keys | `review_id` is not unique (789 ids reused); 547 orders carry 2–3 reviews and 768 carry none | Key on `order_id`, keep the earliest review; no review means `NaN` |
| Sparse review text | `review_comment_title` present for 11.7% of reviews, `review_comment_message` for 41.3% | Use `review_score` only; coverage does not support NLP features |
| Geolocation grain | 1,000,163 rows for 19,010 zip prefixes, including points outside Brazil | Collapse to a median coordinate per prefix; use it for great-circle distance |
| Missing product attributes | 610 products (1.9%) have no category | Label `unknown` rather than dropping orders; impute numerics in-pipeline |
| Coverage ramp | 326 orders before 2017; final weeks truncated mid-cycle | Restrict to 2017-01-01 .. 2018-08-31 (99,091 orders) |
| Portuguese categories | 71 category names | Translate to English for all output; keep the original key |

**Table 11:** Candidate problems against the brief's scoping checklist (Section 4.5), as scored in Phase 1 with untuned probes.

| **Criterion** | **C1 delivery (checkout)** | **C2 low review (delivery)** | **C3 repeat (checkout)** |
|---|---|---|---|
| Derivable target | Yes, from order timestamps | Yes, `review_score` $\leq 2$ | Yes, next order by person |
| No leakage | Yes, checkout regime | Yes, at-delivery regime | Yes, checkout regime |
| Sufficient signal | Regression yes ($-41\%$ MAE); classification weak ($1.6\times$) | Yes, PR-AUC 0.40 vs 0.11 ($3.6\times$) | No, $2.0\times$ at a 1.7% base rate |
| Manageable imbalance | 8.1% late; PR-AUC and thresholds, no resampling | 14.6% low; same handling | 2.6%, too extreme |
| Clear stakeholder | Operations | Customer experience | CRM |
| **Verdict** | **Problem 1** | **Problem 2** | Rejected |

**Table 12:** Data dictionary for the nine source tables. *Null* is measured, not declared: Y means the column actually contains missing values in this extract, with the observed rate given.

| Table | Column | Type | Null | Comment |
|---|---|---|---|---|
| orders | order_id | text | N | Primary key. The analysis grain |
| | customer_id | text | N | **Per-order** key, not a person |
| | order_status | text | N | 8 values; *delivered* for 97.0% |
| | order_purchase_timestamp | datetime | N | The checkout prediction moment |
| | order_approved_at | datetime | Y 0.16% | Post-checkout; excluded as a feature |
| | order_delivered_carrier_date | datetime | Y 1.79% | Post-outcome; splits lead time into seller and carrier share |
| | order_delivered_customer_date | datetime | Y 2.98% | Post-outcome; defines both delivery targets |
| | order_estimated_delivery_date | date | N | Promised at checkout; **admissible**. Date only, 459 distinct values |
| order_items | order_id | text | N | **1:N** — 112,650 rows for 98,666 orders |
| | order_item_id | int | N | Sequence within the order, 1–21 |
| | product_id, seller_id | text | N | Foreign keys |
| | shipping_limit_date | datetime | N | Post-checkout; excluded |
| | price, freight_value | float | N | Per item; summed to order level |
| order_payments | order_id | text | N | **1:N**. One order has no payment row |
| | payment_sequential | int | N | 1–29 |
| | payment_type | text | N | 5 values; credit card 75.4% |
| | payment_installments | int | N | 0–24; $> 1$ for 51.4% of orders |
| | payment_value | float | N | Summed to order level |
| order_reviews | review_id | text | N | **Not unique** — 789 ids reused across orders |
| | order_id | text | N | **1:N**; 547 orders carry 2–3 reviews |
| | review_score | int | N | 1–5; target of Problem 2 |
| | review_comment_title | text | Y 88.3% | Too sparse to use |
| | review_comment_message | text | Y 58.7% | Too sparse to use; no NLP features |
| | review_creation_date | date | N | Post-outcome for the review target |
| | review_answer_timestamp | datetime | N | Post-outcome |
| customers | customer_id | text | N | Joins 1:1 to orders |
| | customer_unique_id | text | N | **The actual person**; 96,096 of them. Grouping key for splits |
| | customer_zip_code_prefix | int | N | Joins to geolocation |
| | customer_city, customer_state | text | N | 4,119 cities, 27 states |
| sellers | seller_id | text | N | 3,095 sellers |
| | seller_zip_code_prefix | int | N | Joins to geolocation |
| | seller_city, seller_state | text | N | 59.7% of sellers are in SP |
| products | product_id | text | N | 32,951 products |
| | product_category_name | text | Y 1.85% | Portuguese; 73 values. Nulls labelled *unknown* |
| | product_weight_g | float | Y 0.01% | Summed to order level |
| | product_length/height/width_cm | float | Y 0.01% | Combined into item volume |
| | product_photos_qty | float | Y 1.85% | Listing-quality proxy |
| geolocation | geolocation_zip_code_prefix | int | N | 19,015 prefixes over 1,000,163 rows |
| | geolocation_lat, _lng | float | N | Collapsed to a median per prefix; out-of-Brazil points dropped |
| category_ translation | product_category_name | text | N | Join key |
| | product_category_name_english | text | N | 71 translations |
| *Derived in src/ — not present in any source file:* | | | | |
| derived | delivery_days | float | Y | Purchase to arrival; target of 1A |
| derived | is_late | bool | Y | Arrival after the promised date; target of 1B |
| derived | distance_km | float | Y 1.2% | Great-circle customer–seller distance |
| derived | estimated_days | float | N | Promised window in days |
| derived | lead_category | text | N | Category of the most expensive item |
| derived | freight_ratio | float | Y | Freight as a share of item value |
| derived | hist_* | float | Y | The eight as-of track records of Section 5.2 |

# C Success criteria registered in advance

Written into `config.EXPECTED_BANDS` before any model was tuned. `evaluate.check_bands()` compares the headline results against them at the end of every training run and writes its verdicts to `report/results/bands.csv`. The alarm is direction-aware: for an error metric the suspicious end of the interval is the low one, for a score the high one, and confusing the two would point the check at the shortfalls and wave the leaks through.

| Problem | Metric | Registered band | Where the range came from | Result |
|---|---|---|---|---|
| 1A | MAE (days) | 3.3 – 4.3 | Phase 1 reported 3.89 untuned | 3.898 |
| 1B | balanced accuracy | 0.58 – 0.72 | Phase 1 reported 0.58 for a direct classifier | 0.705 |
| 1B | ROC-AUC | 0.72 – 0.85 | Untuned scoping probe reached 0.79 for the derived route | 0.782 |
| 2 | balanced accuracy | 0.63 – 0.74 | Phase 1 reported 0.69 | 0.681 |
| 2 | ROC-AUC | 0.70 – 0.79 | Phase 1 reported 0.73 | 0.749 |
| 2 | PR-AUC | 0.34 – 0.48 | Phase 1 reported 0.40 against a 0.11 floor | 0.414 |

# D Hyperparameter search spaces and selected values

| Family | Rung | Classification | Regression |
|---|---|---|---|
| A — Linear | 0 | `LogisticRegression` | `LinearRegression` |
| | 1 | `LogisticRegression` ($\ell_2$, tuned $C$) | `Ridge` (tuned $\alpha$) |
| B — Tree-based | 0 | `DecisionTreeClassifier` | `DecisionTreeRegressor` |
| | 1 | `RandomForestClassifier` | `RandomForestRegressor` |
| | 2 | `HistGradientBoostingClassifier` | `HistGradientBoostingRegressor` |
| C — Ensemble | 0 | `VotingClassifier` (soft, A+B) | `VotingRegressor` (A+B) |
| | 1 | `StackingClassifier` (A+B) | `StackingRegressor` (A+B) |

Every tuned model was searched with `RandomizedSearchCV` over the distributions of Table 13, scored by five-fold expanding-window cross-validation inside the training period: thirty draws per model, except the random forests, which get six because a forest costs one to two orders of magnitude more per fit and Phase 1 had already shown the forest and the boosted trees landing within 0.01 days of each other. Table 14 lists the values that won, which the Phase 2 report referred to but did not print; the deployed ensembles carry exactly their winning members' values, read back from the pickles, and the validation suite asserts it.

**A compromise inside stacking, stated plainly.** `StackingClassifier` and `StackingRegressor` build their meta-features with `cross_val_predict`, which requires the inner folds to *partition* the training set: every row must be predicted exactly once. An expanding window partitions nothing — the earliest rows are never in a test fold — so the closest valid approximation is unshuffled `KFold`, which at least keeps each fold contiguous in time. The meta-learner is consequently fitted on out-of-fold predictions partly made by models trained on later orders. The residual optimism is confined to the meta-learner's coefficients; the base models are refitted on the whole training period afterwards, and the outer protocol scoring the finished ensemble is still the expanding window. It matters less

**Table 13:** Search spaces.

| Model | Draws | Distributions |
|---|---|---|
| `LinearRegression` / `LogisticRegression` | — | No free parameters. Fitted and scored under the same protocol so the row is comparable |
| `Ridge` | 30 | $\alpha \sim \text{loguniform}(10^{-3}, 10^{3})$ |
| `LogisticRegression` ($\ell_2$) | 30 | $C \sim \text{loguniform}(10^{-3}, 10^{2})$ |
| `DecisionTree` | 30 | `max_depth` $\sim$ randint(3, 20) [classifier: (3, 16)]; `min_samples_leaf` $\sim$ randint(5, 200) [classifier: (20, 400)]; `max_features` $\sim U(0.4, 1.0)$ |
| `RandomForest` | 6 | 150 trees; `max_depth` $\sim$ randint(8, 18) [classifier: (6, 16)]; `min_samples_leaf` $\sim$ randint(20, 140) [classifier: (30, 180)]; `max_features` $\sim U(0.15, 0.40)$ |
| `HistGradientBoosting` | 30 | `max_iter` $\sim$ randint(120, 500); `learning_rate` $\sim$ loguniform$(0.02, 0.3)$; `max_leaf_nodes` $\sim$ randint(15, 64); `min_samples_leaf` $\sim$ randint(10, 120); `l2_regularization` $\sim$ loguniform$(10^{-4}, 10)$ |
| `Voting` | — | Weights fixed at 1:2 in the tree's favour. A stated prior, not a tuned parameter: under drift the linear member is materially weaker and an equal blend would be worse than its better half |
| `Stacking` | — | Meta-learner `Ridge`($\alpha = 1$) / `LogisticRegression`($C = 1$); inner CV five contiguous folds — see the note below |

**Table 14:** Selected hyperparameter values, from `report/results/hyperparameters.csv`. Members of the deployed ensembles are the rows marked †; the stacked review model's meta-learner has coefficients 0.405 (linear member) and 5.572 (tree member).

| Problem | Model | Selected values |
|---|---|---|
| 1A | LinearRegression † | no free parameters |
| 1A | Ridge | $\alpha = 659.9$ |
| 1A | DecisionTree | depth 17, min leaf 158, max features 0.897 |
| 1A | RandomForest | 150 trees, depth 14, min leaf 107, max features 0.165 |
| 1A | HistGradientBoosting † | 433 iterations, learning rate 0.1065, 36 leaf nodes, min leaf 117, $\ell_2$ 9.142 |
| 1A | VotingRegressor † | weights 1:2 (linear : tree) |
| 1B | LogisticRegression ($\ell_2$) | $C = 0.004983$ |
| 1B | DecisionTree | depth 9, min leaf 290, max features 0.878 |
| 1B | RandomForest | 150 trees, depth 13, min leaf 132, max features 0.300 |
| 1B | HistGradientBoosting | 426 iterations, learning rate 0.07079, 21 leaf nodes, min leaf 30, $\ell_2$ 8.244 |
| 1B | derived route † | one-parameter logistic on the margin, fitted on validation; threshold 0.11 |
| 2 | LogisticRegression ($\ell_2$) † | $C = 8.431$ |
| 2 | DecisionTree | depth 14, min leaf 211, max features 0.963 |
| 2 | RandomForest | 150 trees, depth 12, min leaf 44, max features 0.349 |
| 2 | HistGradientBoosting † | 208 iterations, learning rate 0.02073, 28 leaf nodes, min leaf 18, $\ell_2$ 0.01789 |
| 2 | StackingClassifier † | meta-learner LogisticRegression($C = 1$); threshold 0.19 |

than it otherwise would because of how the result lands: the ensemble is *flattered* by this arrangement and still does not separate from the boosted trees, which makes the conclusion in Section 7.5 safer rather than weaker.

# E Re-checking a Phase 1 claim about least squares

Phase 1 reported that ordinary least squares was unusable on this design matrix — "rank-deficient, so its coefficients explode". Phase 2 re-ran it four ways. The observation reproduces exactly; the conclusion drawn from it needed correcting.

**Table 15:** Reproducible via `report/results/encoding_diagnostic.csv`. The deficiency is exactly four — one per categorical, because each one-hot block sums to one and is therefore collinear with the intercept.

| Encoding | rank / cols | deficiency | max $\lvert\beta\rvert$ | CV MAE | test MAE |
|---|---|---|---|---|---|
| OLS, full one-hot block | 101 / 105 | 4 | $2.7 \times 10^{13}$ | $3.9 \times 10^{11}$ | 4.181 |
| OLS, one level dropped | 101 / 101 | 0 | 14.7 | 5.242 | 4.181 |
| Ridge, full one-hot block | 101 / 105 | 4 | 9.82 | 5.241 | 4.182 |
| Ridge, one level dropped | 101 / 101 | 0 | 12.5 | 5.240 | 4.183 |

Least squares still returns a minimum-norm solution and its *predictions* are fine — 4.181 days on the test window, indistinguishable from Ridge. What is not fine is the coefficient vector at $10^{13}$, and the cancellation holding it together is fragile enough to fail on one of five expanding-window folds, which is why the cross-validated error is astronomical while the single-split test error looks healthy. Dropping one level per categorical restores full rank, so the linear family is encoded that way; the compromise is that a level unseen in training is scored as the reference level, which affects a handful of test orders with a

payment type absent from training. Once the design is full rank, Ridge and OLS agree to three decimals: regularisation buys numerical stability here, not accuracy.

# F Full result tables

The body reports the models that matter. The complete tables — every model, every metric, cross-validated, validation and test scores side by side — are written to `report/results/`:

| File | Contents |
|---|---|
| `reg_p1a.csv` | Problem 1A: baselines, both ladders, ensembles; MAE, RMSE, $R^2$ |
| `clf_p1b.csv` | Problem 1B: five direct classifiers and two derived routes |
| `clf_p2.csv` | Problem 2: both ladders and both ensembles, all metrics |
| `selection_*.csv`, `cv_folds.csv` | Every candidate on every out-of-sample window, its ranks, and the fold-to-fold spread (Appendix G) |
| `hyperparameters.csv` | Selected values of every searched model and of the deployed ensembles' members (Appendix D) |
| `encoding_diagnostic.csv` | The Phase 1 re-check of Table 15 |
| `cv_protocol.csv` | Temporal against shuffled CV against the forward test |
| `weight_ablation.csv` | Class weighting: discrimination and calibration |
| `calibration_p2.csv`, `threshold_sweep_p2.csv` | Reliability and the operating-point sweep |
| `cost_p2.csv`, `cost_p1b.csv` | Expected net benefit by threshold |
| `perm_p1a.csv`, `perm_p2.csv` | Permutation importance on the test window |
| `segments_*.csv` | Performance by region, category and test month |
| `predictions_*.csv` | Per-order test predictions, from which the figures and the application's replay are drawn |
| `bands.csv` | Headline metrics against the bands registered in advance |
| `capacity_*.csv` | Precision@$K$ and Recall@$K$, pooled and per day, for both flags |
| `lateness_*.csv` | The lateness anatomy of Appendix H |
| `train_val_test.csv` | The deployed models on the training, validation and test periods (Appendix J) |
| `pilot_design.csv`, `pilot_breakeven.csv` | The outreach pilot sizing (Appendix L) |
| `review_timing*.csv`, `story_order.csv` | Review timing against delivery, the reachable share of the call list, the restated value, the on-time check and the example order (Appendix M) |

# G The cross-validation protocol, and every candidate on every window

**Table 16:** One model, three protocols. Raw average precision disagrees; the same numbers divided by each period's no-skill floor do not. Figure 19.

| Protocol | PR-AUC | fold spread | no-skill floor | lift |
|---|---|---|---|---|
| `TimeSeriesSplit`, expanding window | 0.562 | ±0.091 | 0.158 | 3.55× |
| `StratifiedGroupKFold`, shuffled | 0.580 | ±0.006 | 0.158 | 3.66× |
| Forward test window — the truth | 0.414 | — | 0.110 | 3.75× |

Both cross-validated estimates in Table 16 sit about 0.16 above the forward result, which looks like catastrophic optimism — and almost all of it is mechanical. Average

precision's no-skill baseline *is* the positive rate [40], and the low-review rate falls from 15.8% inside the training period to 11.0% in the test window; normalised by that floor the three agree to within 0.2 of a multiple. **Raw PR-AUC is not comparable across periods with different base rates; its lift over the floor is** — and quoting the raw number alone would have supported either a claim of severe overfitting or a claim of none. The shuffled protocol adds a second warning: its fold-to-fold spread is ±0.006 against the temporal protocol's ±0.091, an interval fifteen times narrower on an estimate no closer to the truth.

**Table 17:** Every Problem 2 candidate on every out-of-sample window (PR-AUC), from `report/results/cv_folds.csv`; the standard deviation is over the five expanding folds. "Rank" is the mean rank over the three most recent windows (folds 4, 5 and validation), the selection criterion of Section 6.2. Figure 16. The same table for Problems 1A and 1B is in the CSV.

| Model | fold 1 | fold 2 | fold 3 | fold 4 | fold 5 | val. | mean ± sd | test | rank |
|---|---|---|---|---|---|---|---|---|---|
| StackingClassifier (A+B) *deployed* | 0.413 | 0.537 | 0.617 | 0.660 | 0.614 | 0.409 | 0.568 ± 0.087 | 0.414 | 1.67 |
| HistGradientBoosting | 0.411 | 0.539 | 0.618 | 0.660 | 0.613 | 0.407 | 0.568 ± 0.088 | 0.415 | 2.00 |
| RandomForest | 0.404 | 0.544 | 0.622 | 0.654 | 0.616 | 0.403 | 0.568 ± 0.089 | 0.405 | 2.67 |
| VotingClassifier (A+B) | 0.413 | 0.526 | 0.611 | 0.650 | 0.604 | 0.404 | 0.561 ± 0.084 | 0.407 | 3.67 |
| DecisionTree | 0.380 | 0.510 | 0.595 | 0.640 | 0.592 | 0.389 | 0.543 ± 0.092 | 0.396 | 5.00 |
| LogisticRegression ($\ell_2$) | 0.306 | 0.416 | 0.534 | 0.574 | 0.497 | 0.320 | 0.466 ± 0.095 | 0.291 | 6.33 |
| LogisticRegression | 0.307 | 0.416 | 0.534 | 0.574 | 0.497 | 0.320 | 0.465 ± 0.095 | 0.291 | 6.67 |

Table 17 is what the Phase 2 evaluation asked for. Read down a column, the windows differ far more than the models: every candidate is at its worst on the first fold (early 2017, few orders, a 20% low-review rate) and at its best on the fourth. Read across a row, the three tree-based rungs are within 0.0004 of one another on the cross-validated mean against a fold-to-fold standard deviation of 0.09; the stacked ensemble is ahead of the single boosted model on three of the six windows and behind on three. Cross-validation cannot order them, which is why the rank rule was fixed before any test score was seen and why the deployed choice is reported as a tie resolved by that rule rather than as a win.

# H Lateness anatomy: where the late deliveries came from

Phase 1 read the monthly late-rate series and attributed its spikes to Black Friday and to the May 2018 truckers' strike. The Phase 1 feedback pointed out that the figure does not support the second claim — the largest peaks are February and March 2018, and May is unremarkable — and asked for a source and a more careful link. `src/lateness_anatomy.py` is that analysis, at three resolutions, and Figure 23 draws it.

**Weekly, not monthly.** A ten-day event cannot be resolved by a monthly series. In Monday-to-Sunday purchase weeks the strike of 21 May to 1 June 2018 [4] — nationwide road blockades, during which Correios suspended its time-definite Sedex services and added five business days to standard parcels [45] — is plainly visible, but one week *earlier* than a naive reading expects: orders placed on 14–20 May were promised 19.5 days under normal conditions and were in transit when the roads closed, and 16.0% of them arrived late against a 5.0% average over the preceding five weeks. Of all orders in transit during the strike (4,228), 16.9% were late, against 0.4% for the May orders delivered before it (Table 18). What the monthly series records instead is the platform's reaction: order

volume in the strike week halved, the promised window for orders placed that week rose to 25.3 days and in the following week to 36.8, and the late rate of those orders was 5.5% and 0.4%. That is why June 2018, at 1.4%, is the quietest month in the data.

**February–March 2018 is a different mechanism.** Panel (b) of Figure 23 splits lead time at the carrier handoff. Outside the peak, sellers hand parcels to the carrier in about 3.2 days and the carrier takes 8.7 more; in February–March 2018 the seller's share was unchanged at 3.4 days while the carrier's rose to 13.2, in every customer region. At the same time the promised window was *tightened*, from 26.3 days in January to 22.0 in March — within the São Paulo to São Paulo route alone, from 20.1 to 15.2 days — so the slack that normally absorbs a slow fortnight was removed exactly when it was needed. The late rate peaked at 29.0% in the week of 26 February, the promise was lengthened again from the week of 9 April, and by that week the late rate was back under 5%. The data do not identify the external cause of the carrier slowdown and we do not assert one; what they do establish is that it was carrier-side, network-wide and compounded by a promising decision, which is the careful form of the claim Phase 1 made loosely.

**Table 18:** The cohorts around the strike, from `report/results/lateness_strike.csv`. Carrier days run from the carrier handoff to the doorstep; the promise is the window quoted at checkout.

| Cohort | orders | late | carrier days | promise, days |
|---|---|---|---|---|
| Bought 1–20 May, delivered before the strike | 2,749 | 0.4% | 4.8 | 19.5 |
| In transit during the strike | 4,228 | 16.9% | 13.1 | 25.5 |
| Bought during the strike, 21 May–1 June | 1,502 | 3.5% | 9.2 | 29.4 |
| Bought 2–30 June | 5,912 | 1.4% | 6.7 | 27.5 |

# I Responses to the Phase 1 and Phase 2 feedback

Both rounds of evaluator feedback were treated as specifications. Tables 19 and 20 list each point and the section, table or figure of this report that acts on it.

**Table 19:** Phase 1 feedback (on the literature review, the exploratory analysis and the dashboard) and where this report acts on it.

| Feedback | Response |
|---|---|
| Survey-based perceived quality and platform ratings were treated as one construct | Sections 2 and 1 separate them, cite the self-selection literature [10, 20, 21], and define the target as the platform rating; Section 10 carries the consequence |
| Model choices cited original algorithm papers rather than e-commerce applications; no comparison within the tree family; HistGradientBoosting not justified | Section 2 argues the rungs from the empirical comparisons and from e-commerce applications, including three studies on this dataset, and states why HistGradientBoosting; the ladder results of Section 7 are the within-family comparison |
| Features not supported by literature | Section 5 cites the support for each feature group and uses the correlation matrix as the empirical screen; Appendix K gives a rationale per feature |
| Problem 2 should be evaluated under limited capacity, with Precision@$K$ and Recall@$K$ | Section 7.4, Table 6 and Figure 22; the operating point in Section 9.2 is a capacity; the application returns a top-$K$ triage list and replays any test day |
| Figure 3 attributed the late-rate peaks to the May 2018 strike; the strike needed a source | Section 4.2, Appendix H and Figure 23: the peaks are February–March 2018; the strike is cited [4, 45], shown at weekly resolution, and its effect traced through the promised window |
| Figure 5a omitted São Paulo; "roughly triples"; Figure 8 not referenced | Figure 5a redrawn (the notebook had taken the first eight states alphabetically); growth is about eight-fold (Section 4.1); the correlation matrix is discussed in Sections 4 and 5 |
| Candidate 1 named a stakeholder without saying how the team would use the prediction | Table 2 and Section 9.3: four levers ordered by cost, each with its trigger and the evidence behind it |
| Dashboard inaccessible | Redeployed from the team's current repository under a new URL; Section 8 explains how it stays reachable |

**Table 20:** Phase 2 feedback (on the modelling report) and where this report acts on it.

| Feedback | Response |
|---|---|
| Report CV scores with their fold-to-fold variation for the Problem 2 candidates | Table 5 now carries the mean ± standard deviation over the five folds for every candidate (and Table 3 for the regression); Table 17 and Figure 16 show every candidate on every window; Section 7.3 interprets the spread; the model card shows the same table |
| State the deployed Problem 2 model clearly and consistently; justify the choice | Section 7.5 and Table 7: the StackingClassifier, as pickled, with the justification; the README, the model card and this text are tied to the artifact by a validation check |
| Refer to Figure 3 in the text; the selected hyperparameter values were said to be in Appendix C but were not | Every figure in Appendix O is referred to from the body; the selected values are in Table 14, generated from the results tables and the deployed pickles |
| The expected benefit assumes that contacting a customer prevents the low review; state it as a limitation and propose a pilot or A/B test | Section 10 states it; Section 9.4, Appendix L and Figure 24 design and size the randomised pilot; the model card carries a calculator for it. Appendix M adds the part of the assumption the data *can* check — whether the customer has already written — and restates the value on the reachable share |

# J Training, validation and test side by side

The results tables carry cross-validated, validation and test columns. The in-sample fit on the training period is the missing fourth view, and it is what separates over-fitting from drift: a model that has memorised its training period collapses between training and validation, while a model facing a changed world drops between validation and test, or in step with the base rate. `src/fit_diagnostics.py` scores the three deployed artifacts on all three periods without refitting anything.

**Table 21:** The deployed models on the training, validation and test periods. Thresholds and the margin-to-risk mapping are the ones fitted on the validation period and shipped. Lift is PR-AUC divided by that period's own positive rate.

| Model | Period | $n$ | MAE | RMSE | $R^2$ | | | | |
|---|---|---|---|---|---|---|---|---|---|
| P1A lead time (Voting) | training | 70,617 | 4.718 | 7.807 | 0.408 | | | | |
| | validation | 12,957 | 4.784 | 6.592 | 0.222 | | | | |
| | test | 12,630 | 3.898 | 5.342 | 0.009 | | | | |
| | | $n$ | positives | ROC-AUC | PR-AUC | lift | bal. acc | F1 | Brier |
| P1B late flag (derived) | training | 70,617 | 8.8% | 0.803 | 0.396 | 4.48× | 0.713 | 0.391 | 0.068 |
| | validation | 12,957 | 5.0% | 0.787 | 0.166 | 3.34× | 0.648 | 0.227 | 0.044 |
| | test | 12,630 | 7.4% | 0.782 | 0.214 | 2.89× | 0.705 | 0.235 | 0.066 |
| P2 low review (Stacking) | training | 72,380 | 15.8% | 0.836 | 0.617 | 3.90× | 0.754 | 0.601 | 0.089 |
| | validation | 13,094 | 11.4% | 0.762 | 0.409 | 3.60× | 0.678 | 0.436 | 0.083 |
| | test | 12,855 | 11.0% | 0.749 | 0.414 | 3.75× | 0.681 | 0.470 | 0.078 |

Three readings. **The lead-time blend does not memorise its training period**: its in-sample MAE, 4.72 days, is level with the validation period's 4.78 and *above* the test window's 3.90, and the fall in $R^2$ from 0.41 in-sample to 0.01 on test is the test period's

small variance rather than a large error — the level shift of Section 10, not over-fitting. **The derived late flag is stable**: ROC-AUC 0.80, 0.79, 0.78 across the three periods, while its lift over the floor falls from 4.5× to 2.9× only because the floor itself moves with the late rate. **The stacking ensemble shows the expected, modest in-sample optimism of a tree ensemble**: ROC-AUC 0.84 on the period it was fitted to against 0.76 and 0.75 on the two it never saw, and PR-AUC 0.62 against 0.41 — but validation and test agree with each other (lift 3.6× and 3.75×, Brier improving from 0.089 to 0.078 as the base rate falls), which is the comparison that governs deployment.

# K Feature catalogue

Every feature, with its definition, the reason it is in the model and the moment it becomes knowable. Literature pointers are to the works discussed in Section 2; empirical pointers are to Section 4. All 36 are built by `src/features.py` from the nine tables alone.

**Table 22:** The 36 features by group. "Checkout" features are admissible for both problems; "delivery" features only for Problem 2.

| Feature | Definition | Why it is in the model | Known at |
|---|---|---|---|
| *The promise (2)* | | | |
| `estimated_days` | promised delivery date minus purchase time, days | the platform's own routing knowledge, shown to the customer at checkout; strongest single correlate of lead time after distance ($r = 0.38$) and a decision variable in its own right [9, 41] | checkout |
| `long_promise` | promised window above the pinned median of 23.2 days | flags the routes the platform itself expects to be slow; pinned so serving and training use the same cut | checkout |
| *Basket and money (11)* | | | |
| `n_items`, `n_sellers` | items and distinct sellers in the order | multi-item and multi-seller orders consolidate before shipping; order composition is the standard control in order-level logistics studies [11] | checkout |
| `is_multi_seller` | more than one seller | split shipments wait for the slowest seller | checkout |
| `log_price` | $\log(1 + \text{item total})$ | money is right-skewed; the log stabilises the linear family; price relates to the rating, not to lead time (Figure 8) | checkout |
| `price_per_item` | item total divided by items | separates one expensive item from many cheap ones | checkout |
| `total_freight` | freight paid, R$ | the platform's own pricing of distance and bulk | checkout |
| `freight_ratio` | freight divided by item total | the delivery cost as the customer experiences it, relative to the purchase | checkout |
| `log_weight`, `max_volume_cm3` | $\log(1 + \text{total weight})$; bulkiest item's volume | handling and carrier constraints; physical attributes are standard delivery-time inputs [41] | checkout |
| `total_photos` | listing photos across the items | a listing-quality proxy for the expectation gap behind a low rating | checkout |
| `max_installments` | largest instalment count | payment behaviour and an affordability signal characteristic of this market | checkout |
| *Calendar (6)* | | | |
| `purchase_month`, `purchase_dow`, `purchase_hour` | month, weekday, hour of purchase | seasonality and the weekly and daily rhythm of fulfilment (Figure 4); the usual calendar encodings for transactional data [19, 28] | checkout |

| Feature | Definition | Why it is in the model | Known at |
|---|---|---|---|
| `is_weekend`, `is_business_hours` | weekend; 09:00–18:00 | an order placed outside working hours waits for the next handling day | checkout |
| `is_nov_peak` | purchased on or after 20 November | the Black Friday surge stresses fulfilment: the late rate reached 14.3% in November 2017 (Section 4.2) | checkout |
| *Geography (4)* | | | |
| `log_distance` | $\log(1 + \text{km})$ between postcode centroids | the strongest checkout-time correlate of lead time ($r = 0.39$); distance is the backbone of delivery-time models [30] | checkout |
| `same_state` | seller and customer in the same state | the cheapest route class; only 35.7% of orders | checkout |
| `customer_state` | destination state (27 levels) | regional logistics; rare levels pooled by the encoder | checkout |
| `seller_region` | origin region (5 levels) | origin logistics, collapsed from 27 states so the linear family is not swamped by sparse levels | checkout |
| *Catalogue and payment (2)* | | | |
| `lead_category` | category of the most expensive item (74 levels, rare ones pooled) | product type drives both handling and expectations; the category's effect is otherwise invisible to the model | checkout |
| `lead_payment_type` | payment method of the largest payment | a boleto is paid at a bank and clears days after a card, so handling starts later; the top permutation feature for lead time (Section 9) | checkout |
| *Track records (8), each an as-of aggregate over orders that had closed before this one was placed* | | | |
| `hist_seller_lead_days`, `hist_seller_late_rate`, `hist_seller_low_review_rate` | the seller's mean lead time, late rate and 1–2 star rate over at least five earlier orders | a party's past performance is the standard predictor in supply-chain delay models [1]; the time-respecting form of target encoding [31] | checkout |
| `hist_seller_n` | number of earlier orders the seller estimate rests on | cold start as information: the model can discount a seller it has barely seen | checkout |
| `hist_route_lead_days` | mean lead time on the seller-state to customer-state route | origin–destination history, the route-level analogue of the above [41] | checkout |
| `hist_category_low_review_rate` | the category's past 1–2 star rate | categories differ in how often they disappoint; encoded without seeing the future | checkout |
| `hist_platform_lead_days`, `hist_platform_late_rate` | platform-wide trailing 28-day mean lead time and late rate | the drift tracker: tells the model how fast the network is running right now [16] | checkout |
| *Delivery outcome (3), Problem 2 only* | | | |
| `delivery_days`, `delay_vs_estimate_days`, `is_late` | realised lead time; days late against the promise; late flag | the dominant driver of a 1–2 star review, 9.2% to 54.0% (Section 4.4; Deshpande and Pendem 11); known once the parcel has arrived and forbidden before | delivery |

# L The outreach pilot

The design randomises inside the top-$K$ list, so both arms share the same predicted risk and the comparison is a fair one. Two things follow from the table that a verbal recommendation would miss. The break-even effect grows with capacity — 21.9% at ten contacts a day, 28.4% at twenty, 42.1% at forty — because precision falls as the list is

**Table 23:** Pilot size by assumed effect, from `report/results/pilot_design.csv`. The control-arm rate is the model's precision at that daily capacity on the test window; $n$ is orders per arm for a two-sided two-proportion test at $\alpha = 0.05$; duration assumes $K$ contacts a day split evenly across the arms; net benefit per contact is under the R\$5 : R\$40 assumption. Figure 24.

| $K$/day | control | reduction | treated | $n$/arm (80%) | wks (80%) | wks (90%) | net/contact |
|---|---|---|---|---|---|---|---|
| 10 | 0.572 | 15% | 0.486 | 531 | 16 | 21 | −R\$1.57 |
| 10 | 0.572 | 20% | 0.457 | 299 | 9 | 12 | −R\$0.43 |
| 10 | 0.572 | **21.9% (break-even)** | 0.447 | 250 | 8 | — | R\$0.00 |
| 10 | 0.572 | 25% | 0.429 | 191 | 6 | 8 | R\$0.72 |
| 10 | 0.572 | 30% | 0.400 | 133 | 4 | 6 | R\$1.86 |
| 10 | 0.572 | 40% | 0.343 | 74 | 3 | 3 | R\$4.15 |
| 20 | 0.440 | 20% | 0.352 | 484 | 7 | 10 | −R\$1.48 |
| 20 | 0.440 | **28.4% (break-even)** | 0.315 | 235 | 4 | — | R\$0.00 |
| 20 | 0.440 | 30% | 0.308 | 210 | 3 | 5 | R\$0.28 |
| 20 | 0.440 | 40% | 0.264 | 115 | 2 | 3 | R\$2.04 |

worked deeper, so a contact lower down has less to prevent; a pilot should therefore run at the capacity that would actually be deployed. And the pilot is short at the effect sizes that matter: six weeks detects a one-in-four reduction at ten contacts a day, while an effect too small to pay for itself would take sixteen weeks to confirm, which is itself informative — if the pilot has not found an effect in two months, the policy should not be scaled.

# M When is the review written?

Problem 2 is framed at delivery, before the review is written. Olist's review table carries two timestamps — `review_creation_date`, the day the satisfaction survey was sent, and `review_answer_timestamp`, the moment the customer answered — and comparing the second with the delivery timestamp tests that framing (`src/review_timing.py`). It holds for parcels that arrive on time and fails for most late ones. The survey goes out at delivery or, for a parcel still on the road, once the promised date has passed, so the customer of a late parcel is usually asked to review while still waiting (Table 24).

**Table 24:** Review timing on the 95,561 delivered and reviewed orders of the analysis window, from `report/results/review_timing.csv`. A negative median means the review was answered before the parcel arrived.

| | Late parcels | On-time parcels |
|---|---|---|
| Delivered and reviewed orders | 7,659 | 87,902 |
| 1–2 star rate | 54.0% | 9.2% |
| Survey sent on or after the promised date | 99.0% | 2.1% |
| 1–2 star reviews written before the parcel arrived | **87.3%** | 0.5% |
| Share of all 1–2 star reviews | 33.8% | 66.2% |
| Median answer of a 1–2 star review, days after delivery | −4.5 | 2.1 |

Two consequences follow. **A hit is not always a review that can still be prevented.** At ten contacts a day, 0.572 of contacts reach a would-be low reviewer, but 0.220 reach one who has already written — almost always on a late parcel — and 0.352 one who has not, against 0.093 at random (Table 25). Counting only the reachable, the expected benefit under R\$5 : R\$40 falls from R\$1,466 to **R\$775 per thousand delivered orders** at the same optimum (threshold 0.11), and at ten contacts a day from R\$853 to R\$433. It stays positive: reachable precision is still almost three times the 0.125 break-even. **The action for a late parcel belongs on the promised date, not at delivery.** Customer care can reach the customer as the promise breaks, with an apology

**Table 25:** The review-risk list on the test window, its hits split into customers who had not yet written (reachable) and those who already had, from `report/results/review_timing_capacity.csv`. Precision is the share of contacts that reach a 1–2 star reviewer; the two columns after it sum to it.

| Capacity | precision | reachable | already written | late share of contacts |
|---|---|---|---|---|
| top 1% | 0.705 | 0.419 | 0.287 | 0.403 |
| top 5% | 0.678 | 0.362 | 0.316 | 0.521 |
| top 10% | 0.475 | 0.302 | 0.173 | 0.376 |
| 5 a day | 0.631 | 0.353 | 0.278 | 0.434 |
| 10 a day | **0.572** | **0.352** | 0.220 | 0.362 |
| 20 a day | 0.440 | 0.288 | 0.152 | 0.298 |
| contacting at random | 0.110 | 0.093 | 0.017 | — |

and a new date, and the survey can be sent after the parcel arrives. That contact needs no model, since the calendar identifies the parcels, and it is tested as the second arm of the pilot (Section 9.4). The review-risk list keeps its value for the parcels that arrive on time, which produce two thirds of all low reviews and are reviewed a median 2.1 days after delivery. On those parcels alone the deployed model reaches ROC-AUC 0.711, and its top 5% are 53.8% low reviewers, 5.7 times their base rate, so the list does more than restate lateness (`review_timing_subsets.csv`).

The order that opens the presentation shows the pattern in one case. Bought on 16 July 2018 in São Gonçalo (RJ) from a seller in São Luís (MA), 2,267 km away, and promised in 21 days, it was surveyed two days after the promise passed and rated one star 5.8 days before it arrived, 10.5 days late. It tops the 16 July review-risk list at 0.85; eight of that day's top ten left a 1–2 star review, four of them before their parcel arrived (`story_order.csv`).

# N Contributions

| Member | Contribution |
|---|---|
| Pan Pinyou | Problem scoping and stakeholder framing; feature engineering for the track-record features; business interpretation, the cost analysis and the pilot design; report Sections 1, 5 and 9; presentation slides |
| Li Yudan | Literature review and its revision; modelling pipeline and the family ladders; hyperparameter search; imbalance, calibration and fold-to-fold analysis; report Sections 2, 6 and 7.3 |
| Zhu Gengyu | Data pipeline and the leakage guard; exploratory analysis and the lateness anatomy; cross-validation protocol and the direct-against-derived comparison; dashboard, deployment, tests and reproducibility; report Sections 3, 4, 7.1, 8 and 10 |

All three members reviewed every section and verified the reported numbers against `report/stats_phase3.txt` and `report/stats.txt`.

# O Figures

Figures 2 to 10 are the exploratory analysis, drawn by notebooks 01–04; Figures 11 to 24 are drawn by `src/figures_phase2.py` from the same CSV tables the body's numbers come from, so a figure cannot disagree with a table; Figures 25 to 27 are the deployed application.

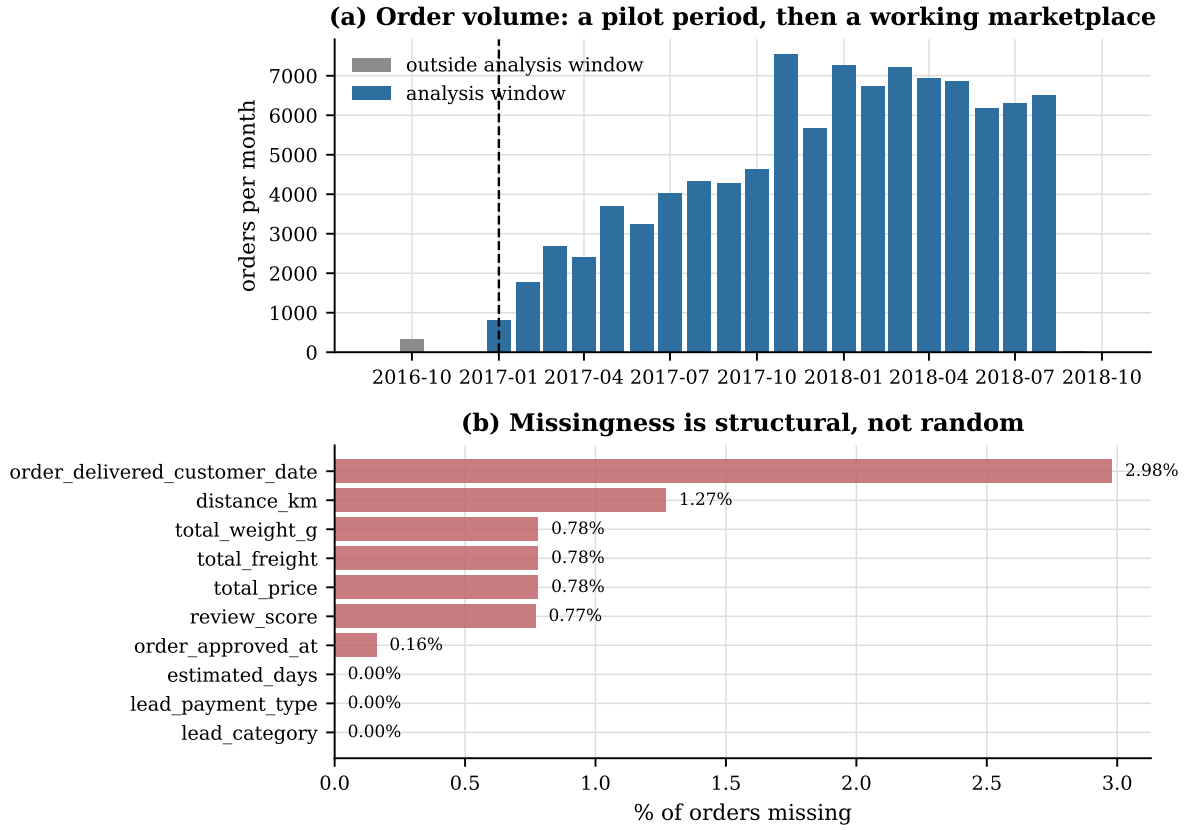



**Figure 2:** (a) Monthly order volume. The pilot period before 2017 contributes 326 orders in total and is excluded from the analysis window. (b) Missingness by column: structural rather than random, dominated by orders that never reached the customer.

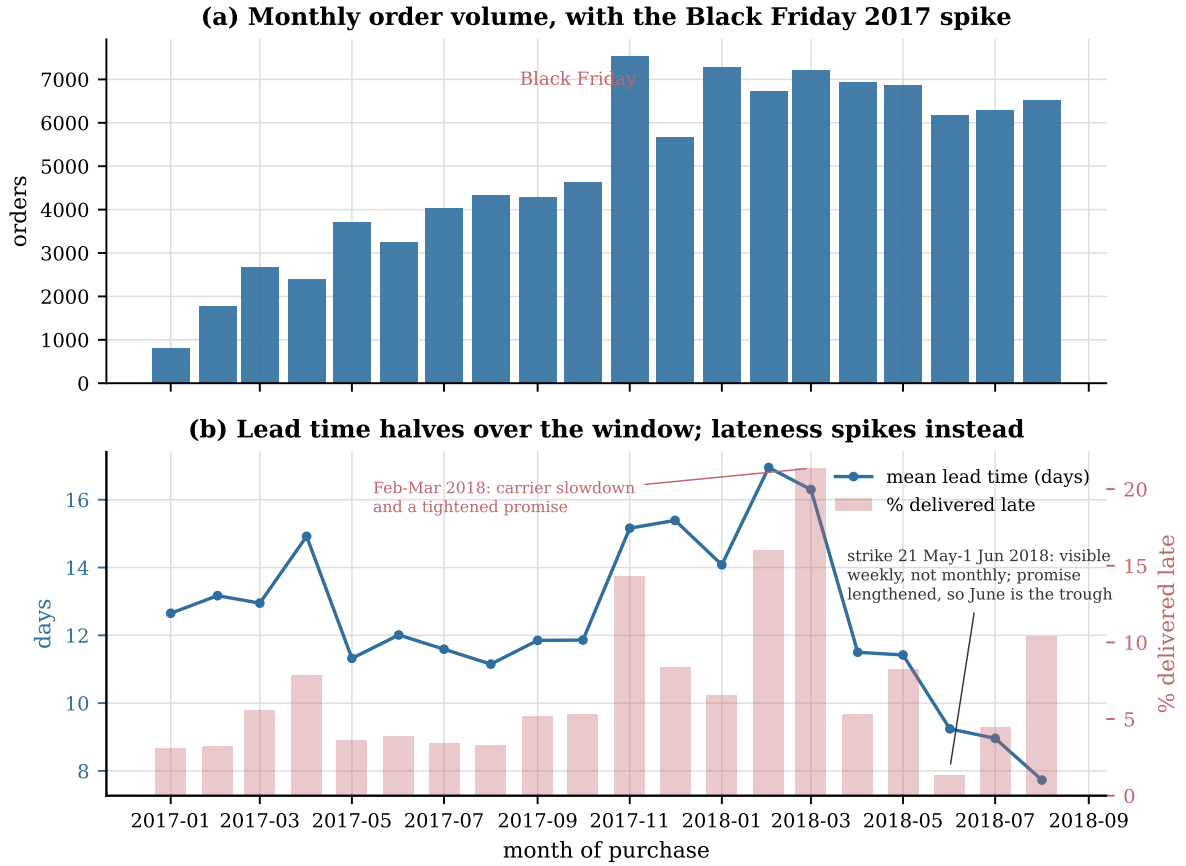



**Figure 3:** (a) Order volume with the Black Friday 2017 spike. (b) The central temporal finding: mean delivery lead time halves across the window (line, left axis) while the late rate is volatile rather than trending (bars, right axis). The spikes are Black Friday (November 2017) and February–March 2018, a carrier-side slowdown that coincided with a tightened promise. The truckers' strike of 21 May–1 June 2018 is not visible at monthly resolution (Section 4.2).

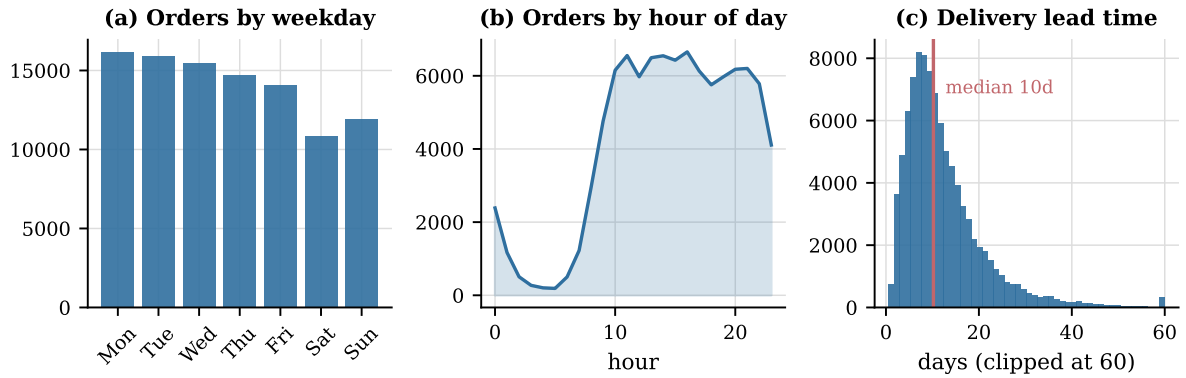



**Figure 4:** Purchase rhythm and the lead-time distribution. Ordering is weekday-heavy and peaks at 16:00; lead time is right-skewed with a median of 10.2 days and a long tail.

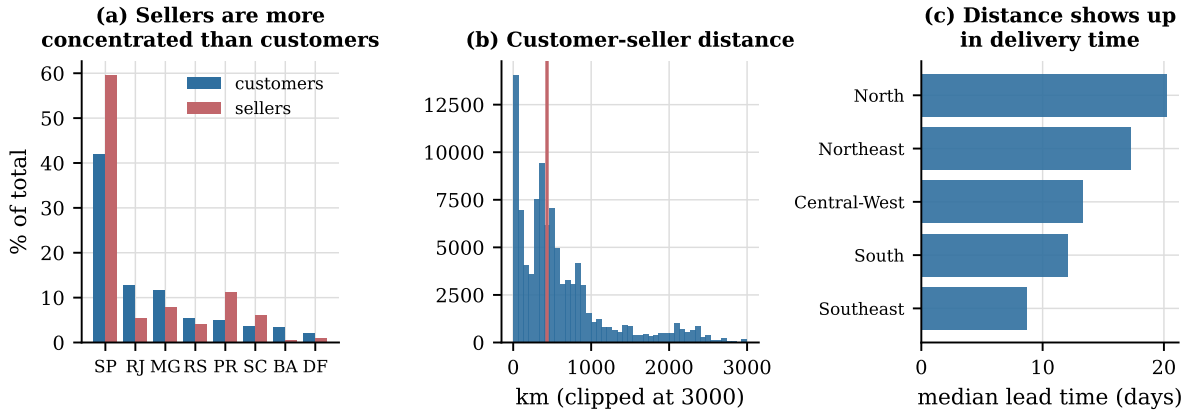



**Figure 5:** (a) Sellers are far more concentrated in São Paulo than customers are, so most orders must cross the country. (b) Customer–seller distance. (c) That distance shows up directly in regional delivery times.

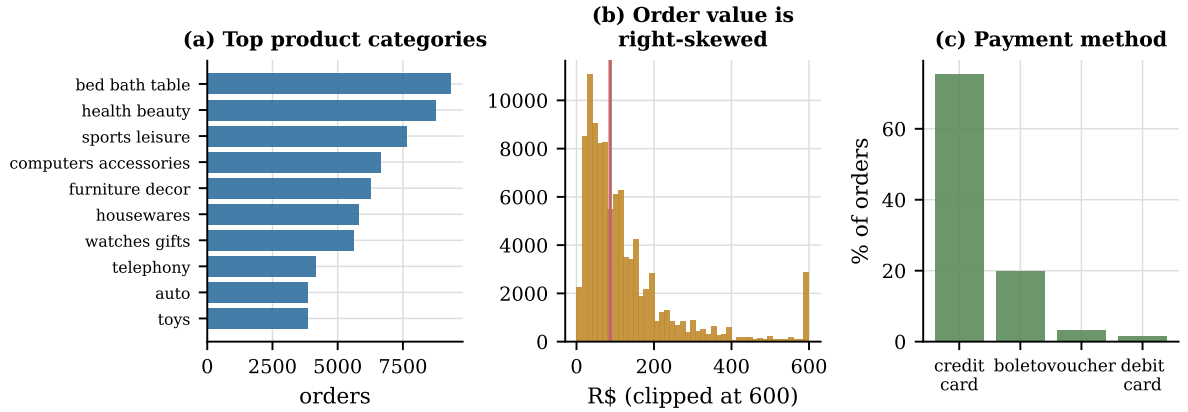



**Figure 6:** (a) Demand is long-tailed across 74 categories. (b) Order value is strongly right-skewed. (c) Credit card dominates, usually in instalments.

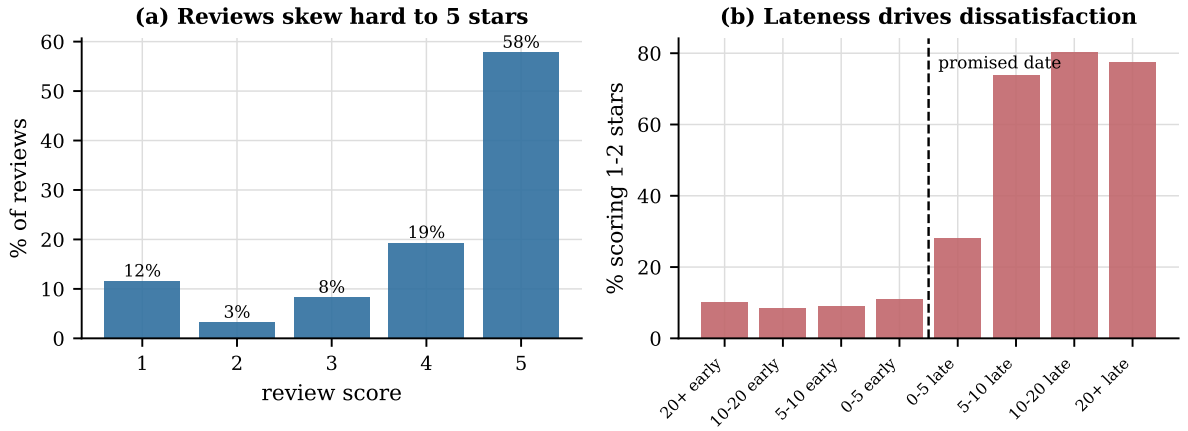



**Figure 7:** (a) Reviews are severely imbalanced towards five stars. (b) The strongest relationship in the dataset: the 1–2 star rate rises monotonically with lateness, from 9.2% for on-time orders to 54.0% for late ones.

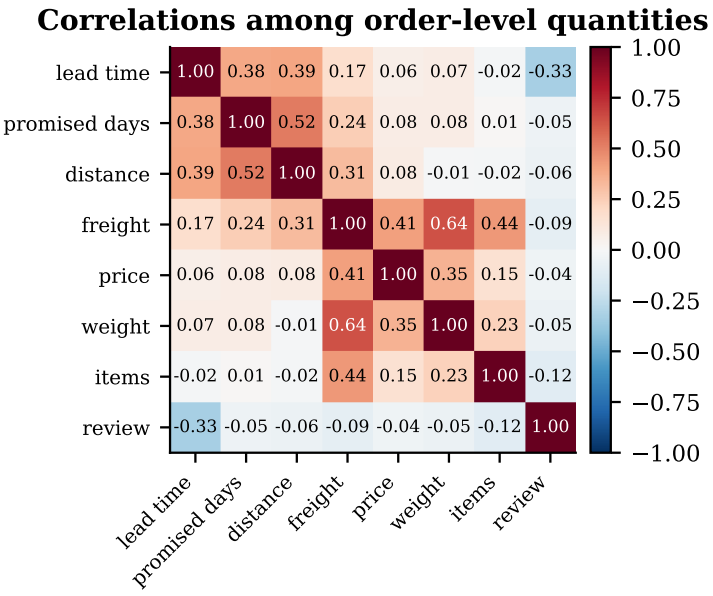



**Figure 8:** Correlations among order-level quantities. Distance and the platform's own promise are the strongest checkout-time correlates of lead time; the review score moves with lead time and not with price.

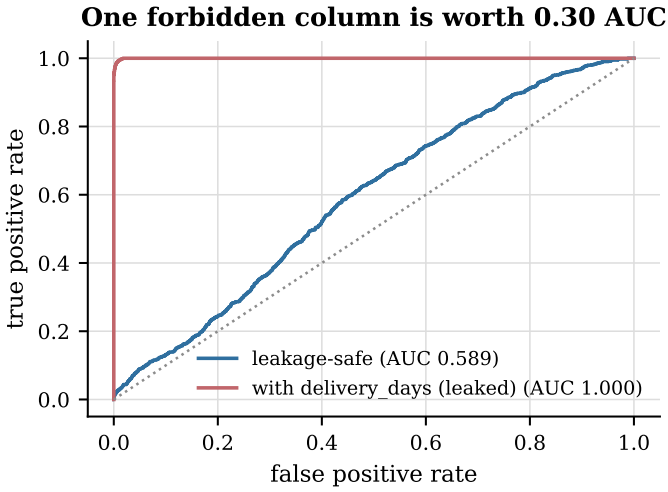



**Figure 9:** What one forbidden column is worth. Adding the realised delivery time to the late-delivery model takes ROC-AUC from 0.589 to 1.000 — the model is reading the answer, not predicting it.

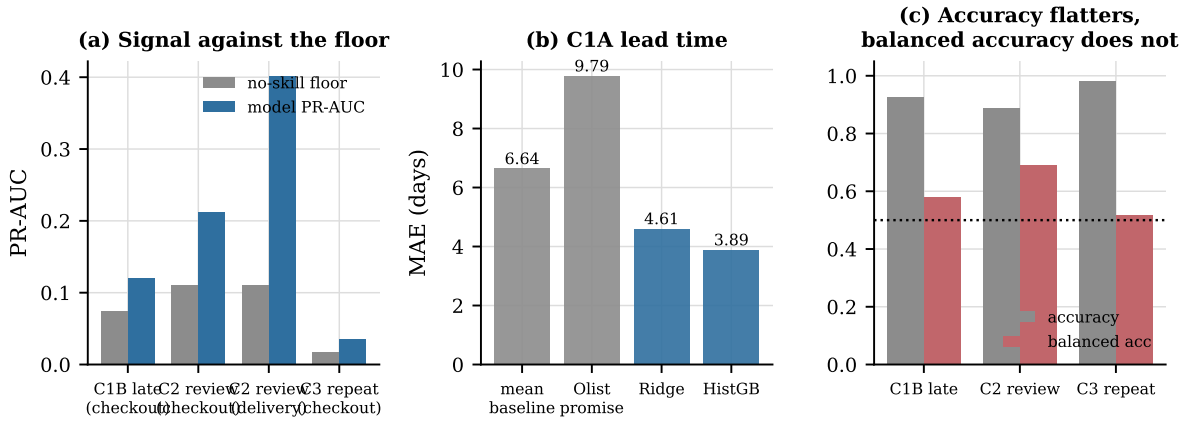



**Figure 10:** Candidate comparison from the scoping phase. (a) PR-AUC against the no-skill floor for each candidate and prediction moment. (b) Lead-time MAE: the platform's own promise is worse than a constant. (c) Raw accuracy flatters every imbalanced target; balanced accuracy does not.

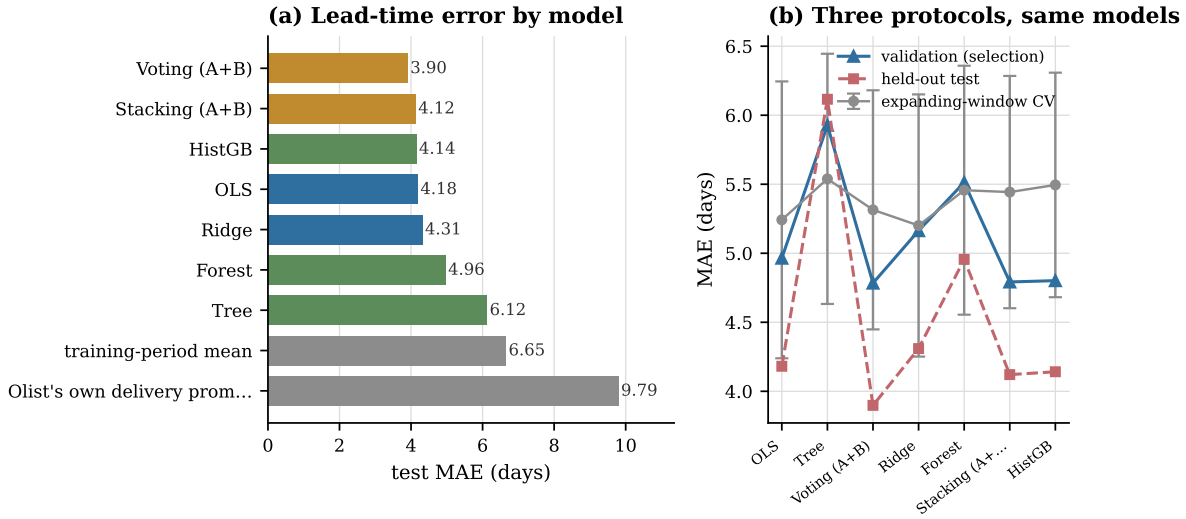



**Figure 11:** Lead-time error by model, and the same models under three protocols. Panel (b) is the argument of Section 6.2: expanding-window cross-validation does not order the candidates the way the validation period and the forward window — which agree with each other — do.

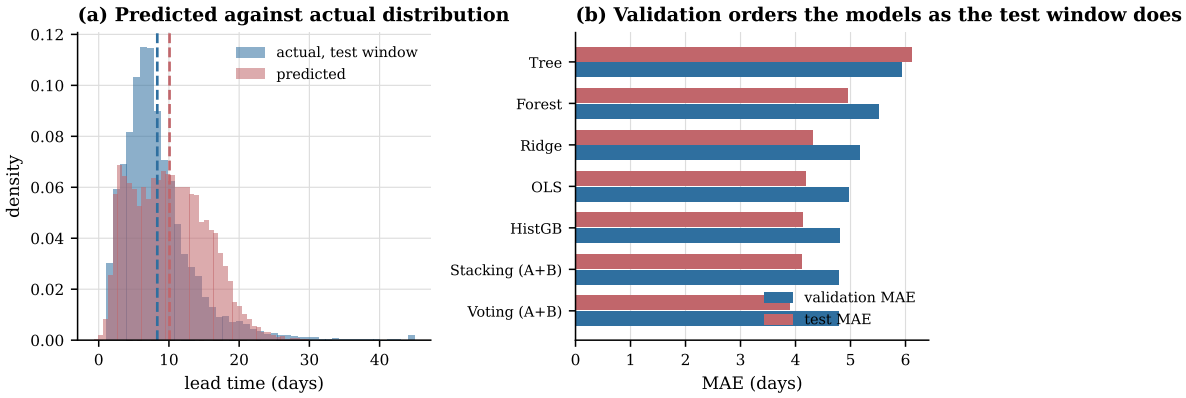



**Figure 12:** Why blending a linear model with a boosted one helps under drift. A tree cannot predict outside the range of the targets it was fitted on; lead times in the test window are systematically shorter than anything common in training. Panel (b) shows the validation period ordering the models as the test window does.

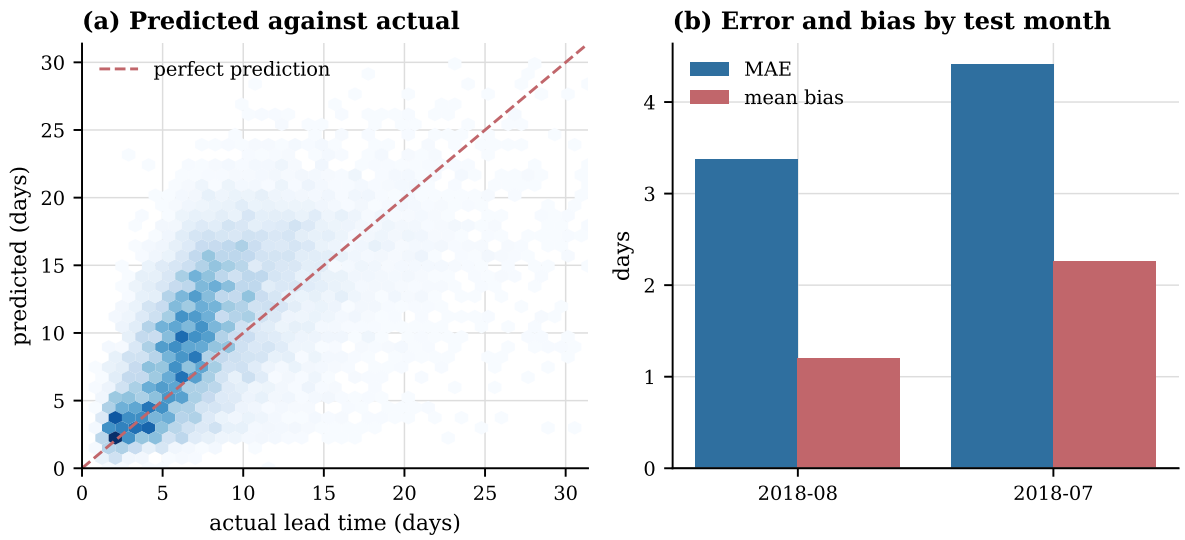



**Figure 13:** Where the regression is wrong. The bias in panel (b) is positive in both test months: a model fitted on a slower past over-predicts how long a parcel will take.

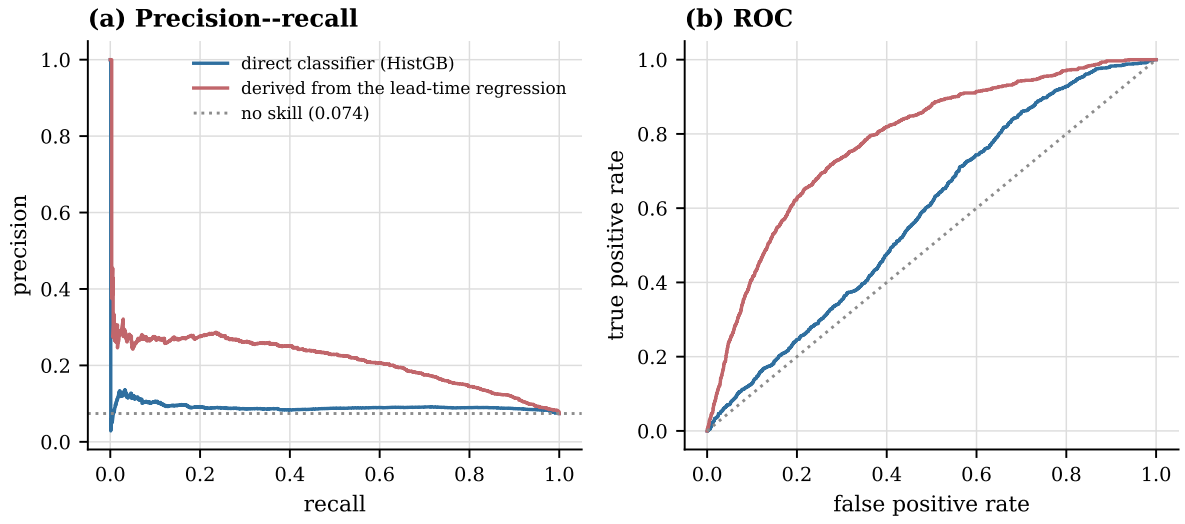



**Figure 14:** Problem 1B. The flag derived from the lead-time regression against the best purpose-built classifier, on the forward test window.

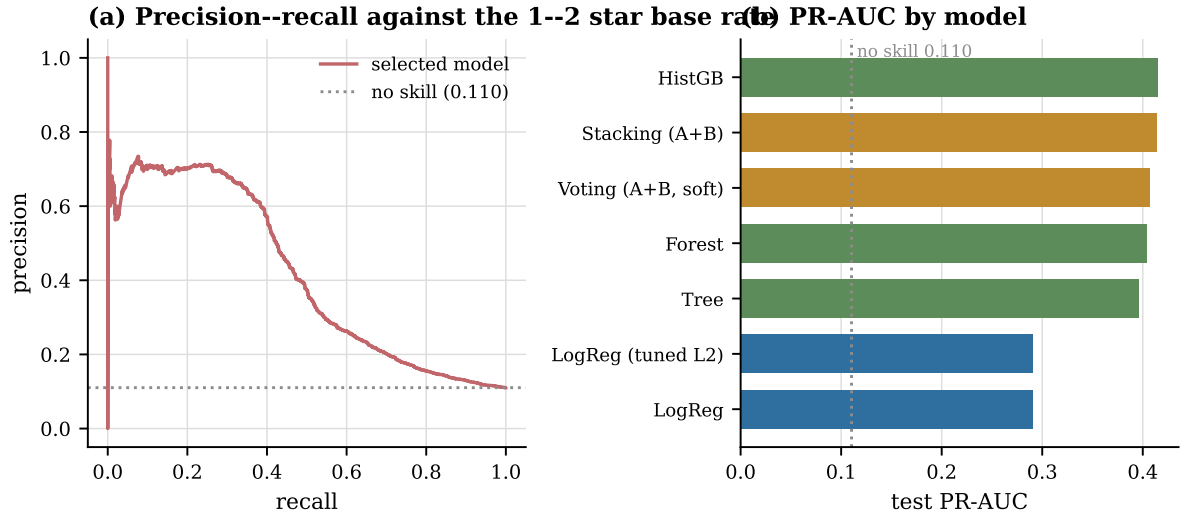



**Figure 15:** Problem 2 against its no-skill floor, and PR-AUC by model. Most of the achievable signal is already available from a single depth-limited tree.

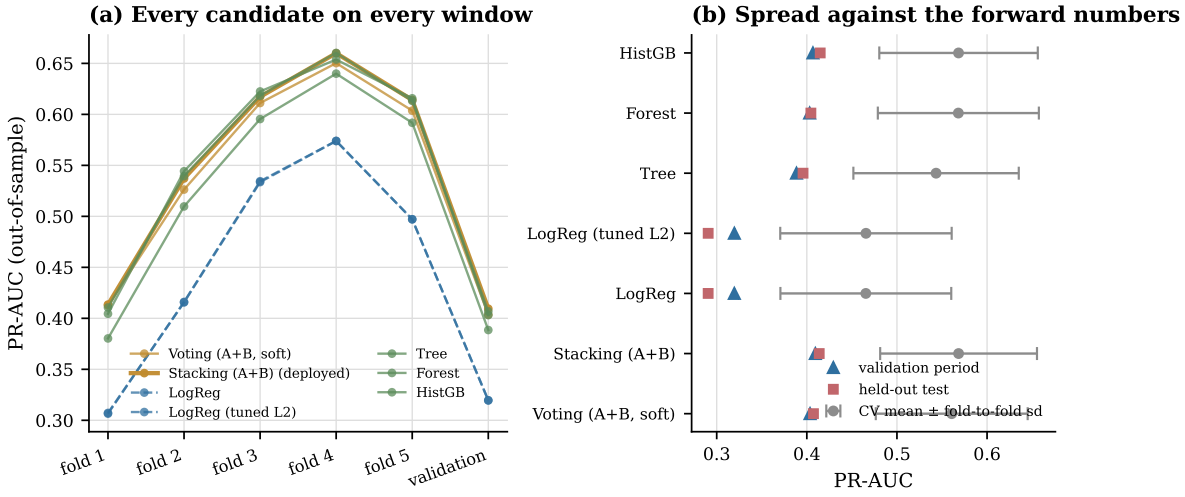



**Figure 16:** Fold-to-fold variation of every Problem 2 candidate (Table 17). (a) Each candidate's PR-AUC on the five expanding folds and the validation period: the windows move together, because their difficulty differs far more than the candidates do. (b) The cross-validated mean with its fold-to-fold standard deviation beside the validation and test scores: the spread is about 0.09 against a gap of under 0.001 between the best candidates.

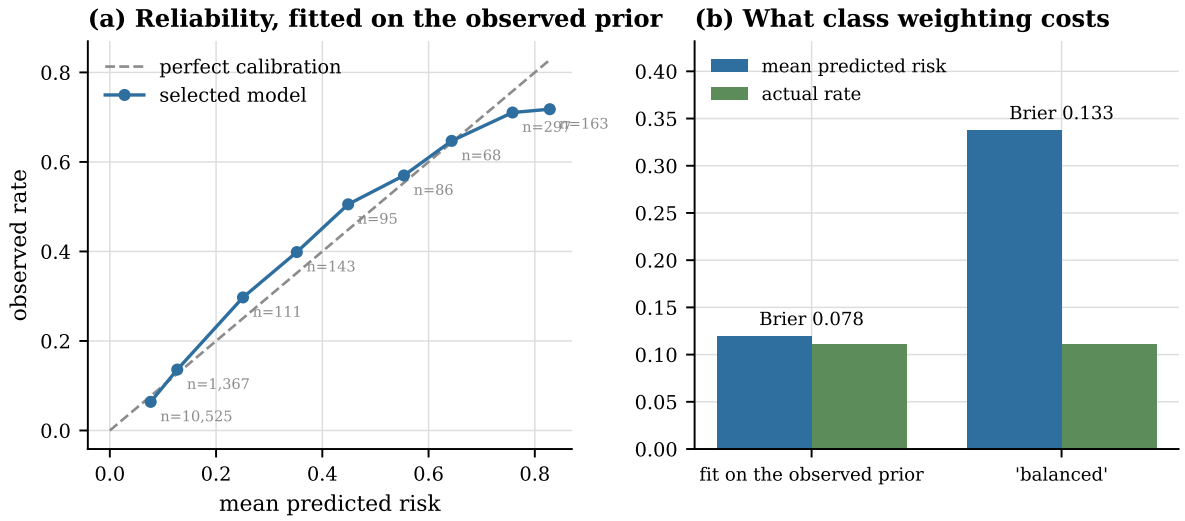



**Figure 17:** Calibration, and what class weighting costs. Weighting the loss leaves ranking unchanged and inflates mean predicted risk threefold against the actual rate.

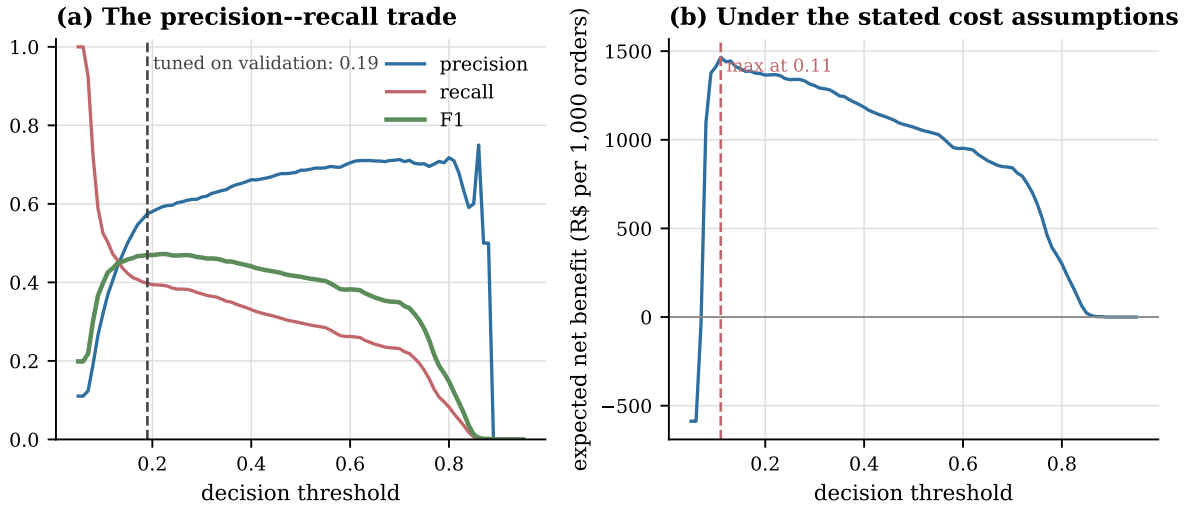



**Figure 18:** Choosing the operating point for the review model, and what it is worth under the stated cost assumptions.

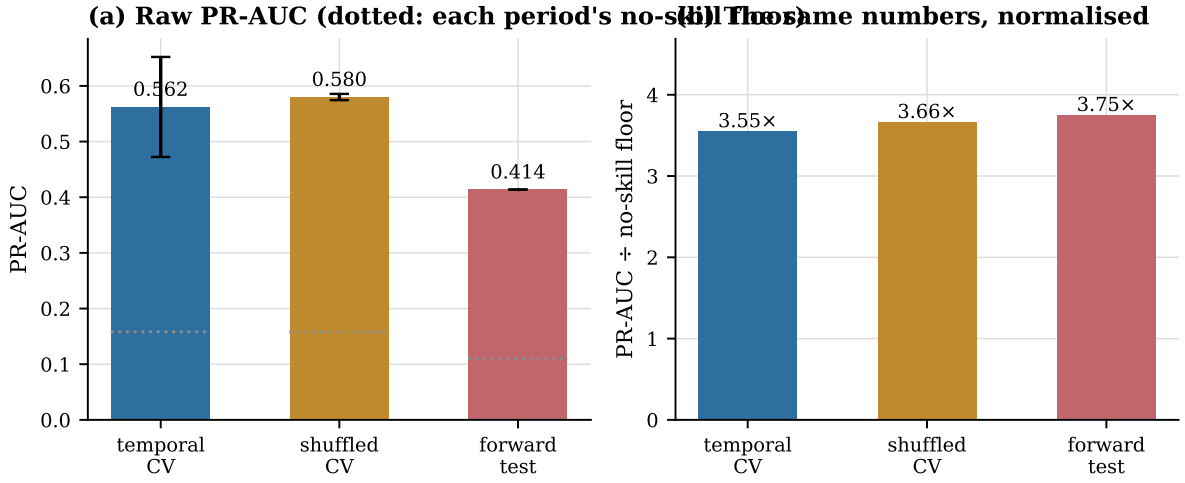



**Figure 19:** One model, three protocols. Raw average precision disagrees across periods; divided by each period's no-skill floor the three agree to within 0.2 of a multiple.

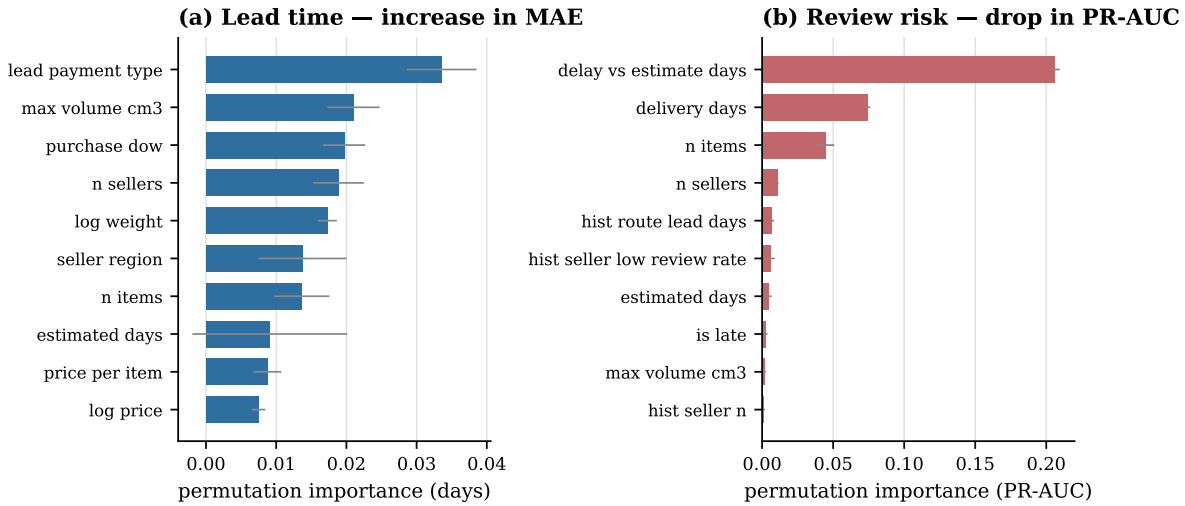



**Figure 20:** Permutation importance on the test window. The review model rests on two features; the lead-time model on none in particular, which is a property of correlated predictors as much as of the model.




**Figure 21:** Performance by customer region. The review model is stable across all five; the regression is not, and degrades exactly where deliveries are longest.

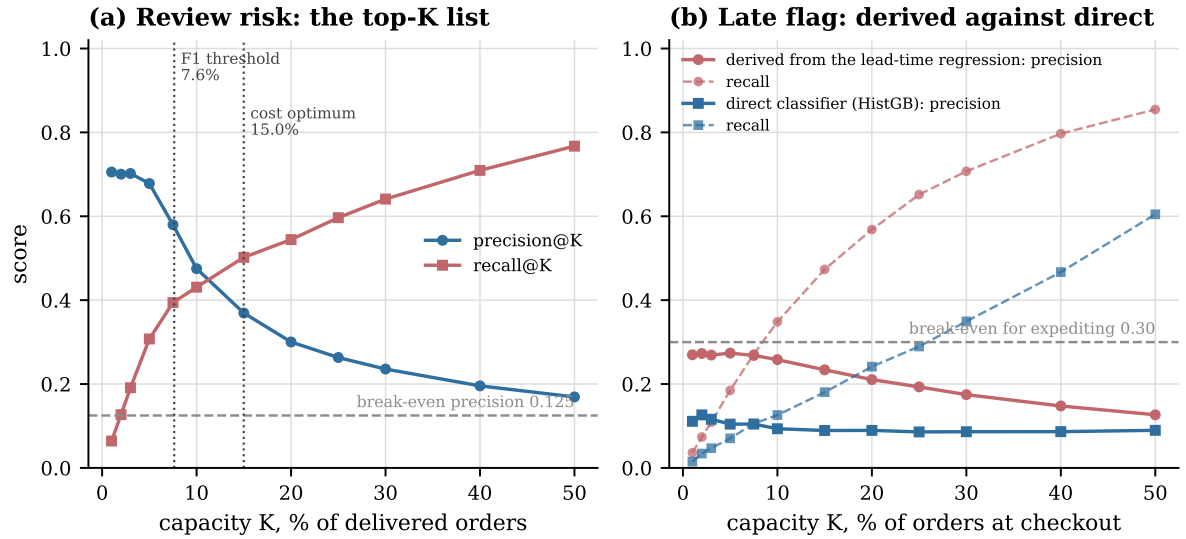



**Figure 22:** Precision and recall when only the top $K$ can be acted on (Section 7.4). (a) The review-risk list, with the report's two threshold operating points placed on the capacity axis; precision stays above the break-even at every capacity. (b) The late flag: the derived route beats the direct classifier at every $K$ and never reaches the break-even an expensive expedite would need.

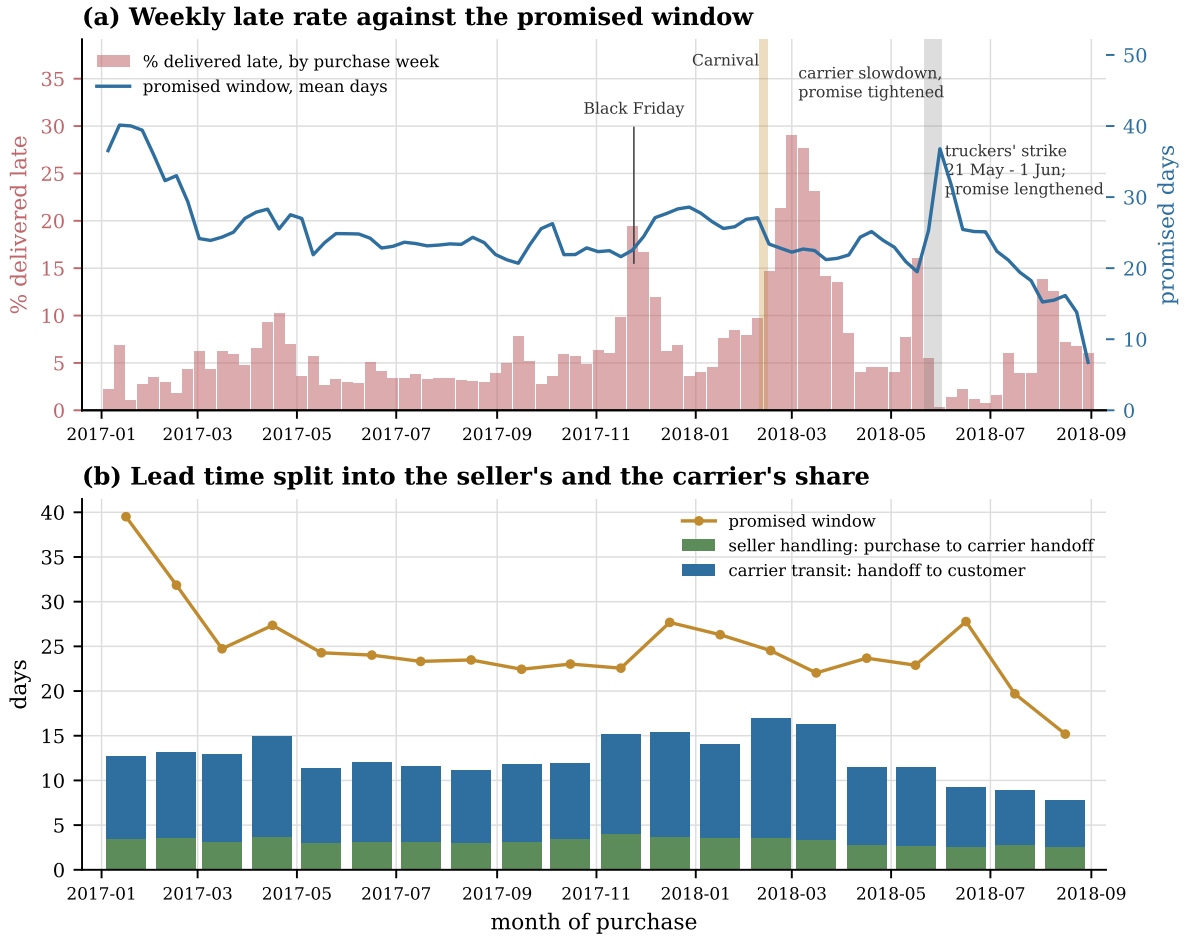



**Figure 23:** Where the late deliveries came from (Appendix H). (a) Weekly late rate against the promised window: the February–March 2018 peak coincides with a tightened promise, and the strike of 21 May–1 June 2018 (shaded) hits the week promised before it, after which the promise is lengthened and the late rate collapses. (b) Lead time split at the carrier handoff: the peak is carrier transit, not seller handling.

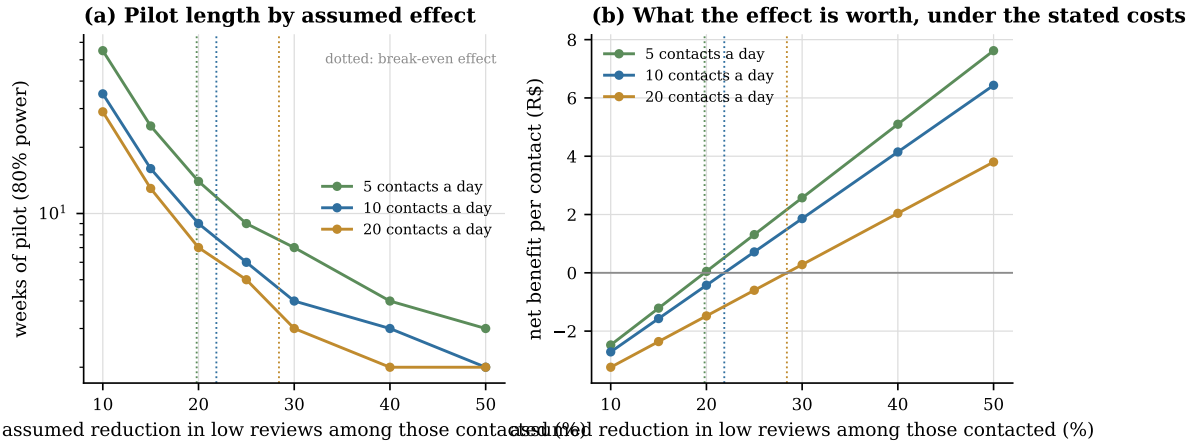



**Figure 24:** Sizing the outreach pilot (Section 9.4). (a) Weeks needed, at 80% power, to detect a given reduction in low reviews among the customers contacted, for three daily capacities; the dotted lines mark each capacity's break-even effect. (b) The net benefit per contact that reduction would imply under the stated costs.

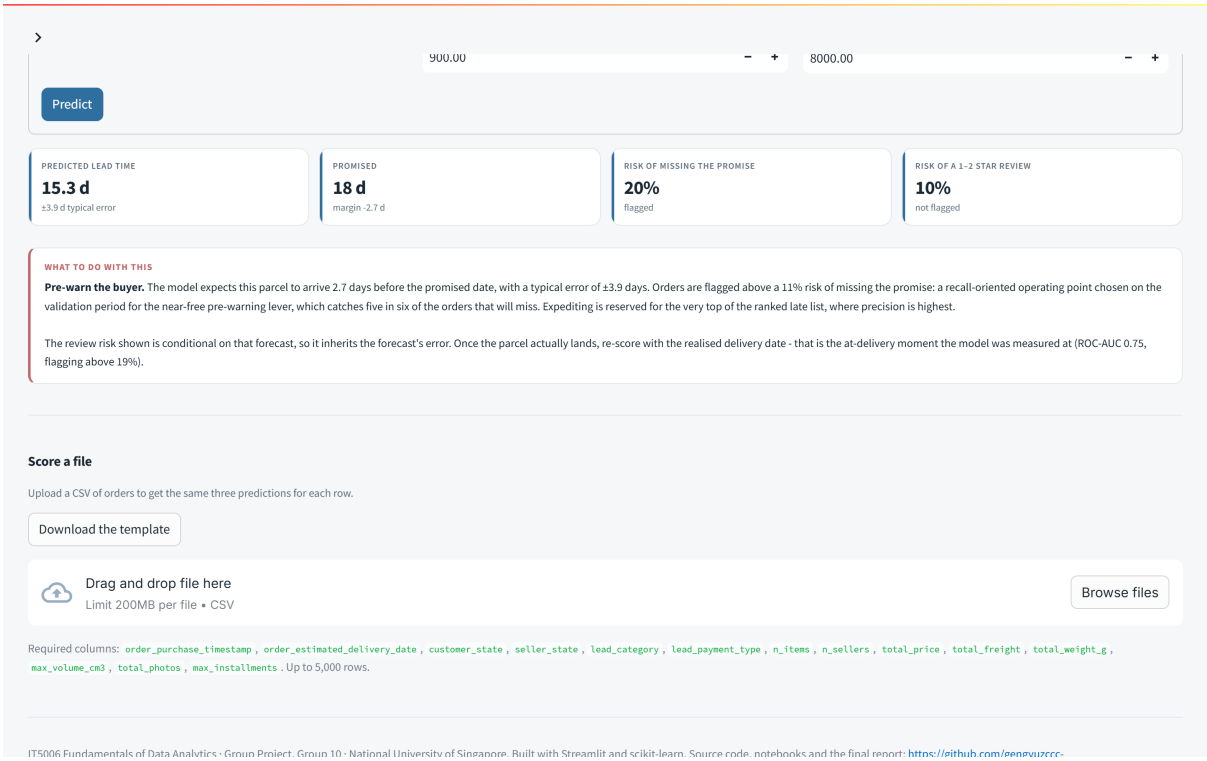



**Figure 25:** The deployed application, Predict tab: one order scored at checkout — expected lead time, the risk of missing the promise against the tuned threshold, and the resulting review risk — with the batch scorer and the top-$K$ triage list below.

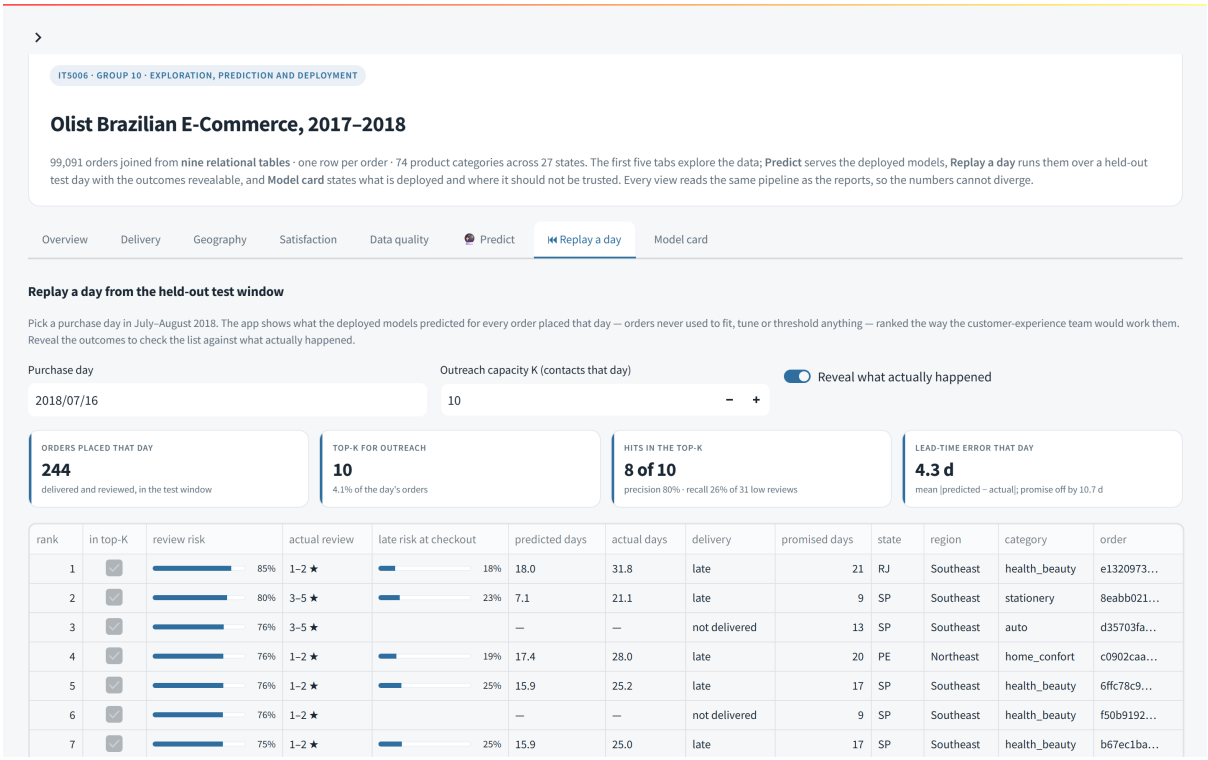



**Figure 26:** The deployed application, Replay a day: the held-out orders placed on 16 July 2018 ranked by review risk with a capacity of ten, outcomes revealed. Eight of the ten would have reached a customer who left a 1–2 star review; the right-hand panel shows the same capacity's precision on every day of the test window.

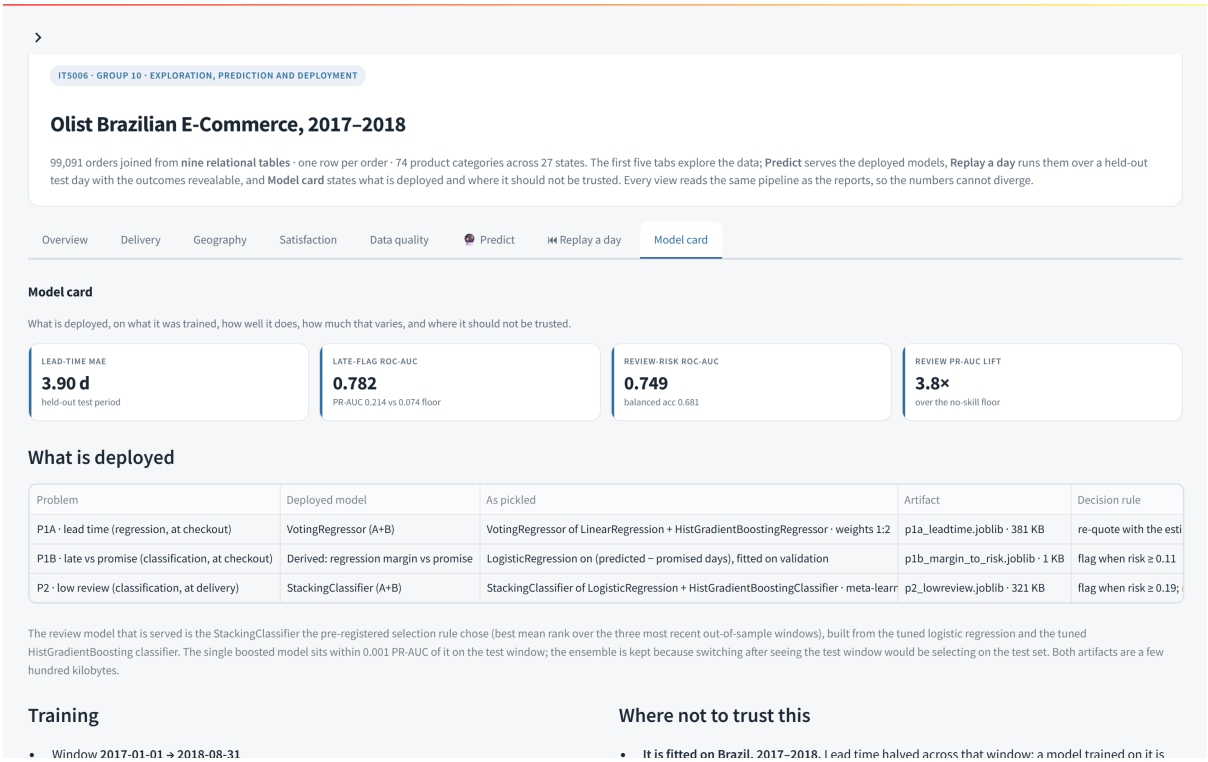



**Figure 27:** The deployed application, Model card: the estimators as pickled, the training protocol, where the models should not be trusted, and the fold-to-fold table.

# AI Tool Declaration

We used OpenAI's ChatGPT and Anthropic's Claude to generate and validate ideas, improve expression and refine our writing. We also used the models to debug our code. All analytical decisions, results and conclusions were reviewed and verified by the team against the underlying data, and we are responsible for the content and quality of the submitted work.